[Full Version] On Optimizing Route-Responsive Urban Tree Placement in City-Scale Pedestrian Networks

Tianyou Gao
Kyoto University
Kyoto, Japan
gao.tianyou.66z@st.kyoto-u.ac.jp

Takayuki Ito
Kyoto University
Kyoto, Japan
ito@i.kyoto-u.ac.jp

Abstract

Urban tree siting is widely studied in environmental science and urban planning, yet is comparatively less explored from an algorithmic optimization perspective. We introduce a modeling framework for shade-aware tree siting and formulate the Route-responsive Urban Tree Optimization (RUTO) problem, which selects k planting sites to maximize pedestrians' expected utility under random origin–destination demand and utility-maximizing path choice. RUTO exhibits an inherent bilevel structure, where placement decisions induce route responses. Although the resulting objective is non-submodular, we show that it admits submodular upper and lower bounds that capture its max-coverage structure. Building on these bounds, we instantiate a sandwich approximation (SA) framework that yields solution-dependent approximation guarantees. We also propose proxy objectives that provide faster heuristics than SA-based methods, albeit without formal guarantees. Experiments on synthetic grids and real-world pedestrian networks show that our SA-based algorithms and proxy heuristics match a direct greedy baseline in objective value while being up to two orders of magnitude faster.

CCS Concepts

• **Theory of computation** → **Facility location and clustering**;
• **Information systems** → *Geographic information systems.*

Keywords

Urban Tree Siting, Spatial Decision Support, Decision-Dependent Optimization, Approximation Algorithm

ACM Reference Format:
Tianyou Gao and Takayuki Ito. 2026. [Full Version] On Optimizing Route-Responsive Urban Tree Placement in City-Scale Pedestrian Networks. In *Proceedings of the 32nd ACM SIGKDD Conference on Knowledge Discovery and Data Mining V.2 (KDD '26), August 09–13, 2026, Jeju Island, Republic of Korea.* ACM, New York, NY, USA, 16 pages. https://doi.org/10.1145/3770855.3817687

1 Introduction

Urban trees are widely recognized to improve pedestrian-level thermal comfort through shading and evapotranspiration, which is supported by empirical studies and syntheses [6, 18]. As an important research topic in environmental science, urban tree siting has motivated a growing line of work on decision support for selecting planting locations with microclimate benefits, where the objective focuses on cooling or comfort indicators [8, 12, 33]. However, the core problem, namely selecting tree locations in urban settings to maximize a well defined objective such as pedestrian comfort, has been studied mainly within environmental science, and the resulting literature remains fragmented and task-specific. To the best of our knowledge, there is no unified algorithmic formulation of this problem, even though it aligns closely with classical operations research models (e.g., covering, facility location, routing).

To fill this gap, we formally introduce a general problem, *Route-responsive Urban Tree Optimization (RUTO)*, that models the tree placement task in urban environments. RUTO converts application specific procedures into an explicit optimization problem on a grid network, i.e., a rasterized representation of a real urban area derived from geographic information system data. Given a budget of k trees, the goal is to select a set of nodes as planting sites to maximize the expected utility of pedestrians, which we model as agents whose *Origin–Destination (OD)* pairs are sampled from an arbitrary distribution. Figure 1 illustrates the RUTO problem.

We first outline the design of RUTO and connect it to well known optimization problems on networks. We define pedestrian utility in a transparent manner that accommodates multiple utility functions, and we model it as typically increasing with the proximity of trees along the chosen route while decreasing with route length. Since the route is not fixed in advance and the placement of trees influences route choice, the objective is decision-dependent. Consequently, RUTO can be viewed as a special case of the bilevel network design problem [17], where the upper level decides tree locations and the lower level represents best response routing. At the same time, there is a close relation to the *Maximum Coverage Problem (MCP)* [5], because selecting the top-k planting sites resembles selecting a top-k cover set, e.g., tree influence covers nodes along routes. The MCP is NP-hard and its objective is submodular, so a simple greedy algorithm achieves a $(1 - 1/e)$ approximation [28]. However, we prove that the RUTO objective is non-submodular because routing is decision-dependent and the utility model is general, hence the $(1 - 1/e)$ guarantee does not apply. Nevertheless, the naive greedy algorithm remains empirically effective, i.e., sample a large set of pedestrian agents and add the tree node with the largest marginal gain at each step, yet each addition requires updating best-response routes for many agents, resulting in high computational cost.

To secure approximation guarantees and reduce computational cost, we adopt the *Sandwich Approximation (SA)* strategy proposed by Lu et al. [22], which is originally developed for the influence maximization problem in online social networks. This strategy requires us to find submodular upper and lower bounds for the non-submodular RUTO objective and provides a solution-dependent

KDD '26, Jeju Island, Republic of Korea

ACM ISBN 979-8-4007-2259-2/2026/08
https://doi.org/10.1145/3770855.3817687

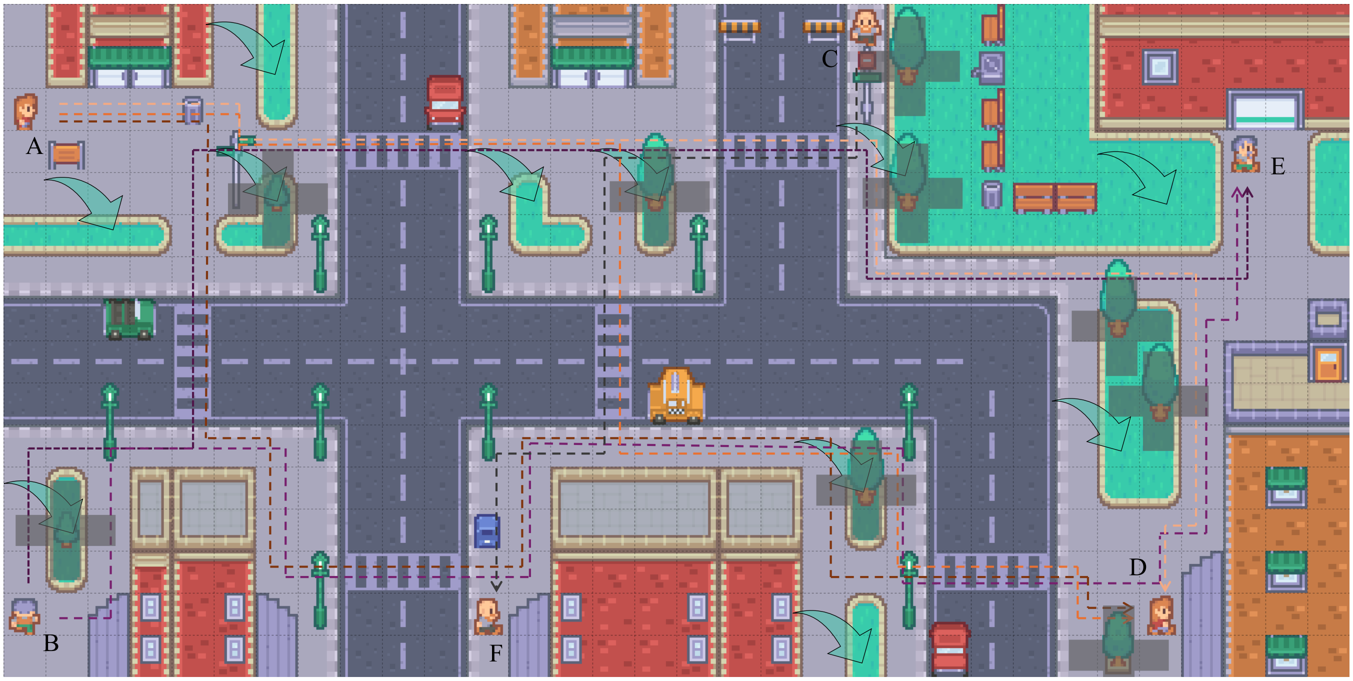


Figure 1: Illustration of the RUTO problem on a rasterized two dimensional city grid. For clarity, we render the map in a pixel art style using assets from RPG Urban Pack by Kenney, licensed under Creative Commons CC0 1.0 Universal. Candidate planting sites are indicated by green arrows. Each planted tree produces a cross shaped shade footprint in this example, while in RUTO the shade kernel is not restricted and can be specified in a more general form. Given a distribution of pedestrian OD-pairs, each pedestrian, modeled as an agent, chooses a path that maximizes its utility, which increases with the proximity and abundance of shaded nodes along the route and decreases with path length. We do not fix a particular mathematical form of utility function, and we allow a broad family that reflects application-specific factors, e.g., ambient temperature, solar irradiance, canopy size and density, and the exact specification is inherently interdisciplinary and therefore outside the scope of this paper. We show three example agents with pairs A to D, B to E, and C to F, and dashed lines denote candidate routes. The example highlights decision dependent routing, since agents adapt their chosen routes to shading. The optimization task is to select k planting sites from the candidates so that the aggregate utility is maximized.

approximation guarantee. In this paper, we design two submodular upper bounds and three submodular lower bounds that can be estimated efficiently, i.e., they avoid solving the inner best response for pedestrian agents, which in turn enables fast greedy selection. Beyond these bounds, we develop two proxy objectives that can be estimated efficiently and that are empirically effective, and we use them to perform greedy selection for RUTO.

We conduct extensive experiments on synthetic two dimensional grid graphs and on multiple real-world pedestrian networks. Through comprehensive scalability analyses and empirical studies, we demonstrate that the proposed SA-based algorithms and proxy-based heuristics are effective and efficient. In particular, our methods achieve speedups of up to 100× compared with the lazy greedy baseline method.

Our contributions are as follows:

- We formulate RUTO, a tree placement problem on urban pedestrian networks where planting decisions shape route utilities and agents choose utility maximizing paths in response, as a general bilevel optimization problem.
- We prove that RUTO is NP-hard and that its objective function is non-submodular, although it is closely related to the classical MCP.
- We propose a solution-dependent approximation algorithm using the SA strategy, where two upper bounds and three lower bounds are constructed. Furthermore, we propose two efficient heuristics for RUTO.
- We conduct extensive experiments on synthetic grids and real-world pedestrian networks, demonstrating the effectiveness and efficiency of the proposed methods.

Our code is available at https://github.com/TianyouG/RUTO.

2 Related Work

We review two complementary lines of research, i.e., environmental science on the application side and computer science on the modeling side.

Environmental science. Empirical and synthesis studies document that urban trees improve pedestrian thermal comfort through shading and evapotranspiration [6, 18]. Decision-support studies then optimize planting locations for microclimate benefits, where objectives target cooling or comfort indicators and where methods rely on task-specific simulation models or design tools [8, 12, 16, 21, 33]. These works motivate our problem, whereas they are typically application-specific and do not provide a systematic analysis of the underlying algorithmic frameworks.

Computer science. At a high level, our formulation is a bilevel network design problem, in which a planner modifies network attributes and agents choose utility-maximizing paths [3, 9, 17]. In our case, path utility sums benefits over the visited nodes and subtracts a travel cost, which is related to the orienteering problem [10, 11, 32]. Furthermore, RUTO is closely related to the *Maximal Covering Location Problem (MCLP)*, which seeks facility locations that maximize covered demand under a budget [5, 25, 26], whereas coverage in RUTO arises from decision-dependent route utilities rather than a fixed radius rule. Abstracting away geometry, the MCP captures the same combinatorial core, where each candidate facility corresponds to the subset of demand nodes it covers [15]. MCP arises in many network optimization tasks, including sensor placement for traffic management in road networks [19, 27] and influence maximization for information diffusion in online social networks [14, 20]. Beyond classical MCLP and MCP, bilevel MCLP has also been studied in task-specific facility-location settings [1, 2, 4, 7, 23], yet the tree-placement scenario considered in RUTO remains unaddressed. On the algorithmic side, recent work has also expanded the scope of submodular optimization to various settings [24, 29–31]. However, these methods are not directly applicable to RUTO, because the distinctive feature of our problem is its bilevel, route-responsive structure rather than a given submodular objective.

3 Problem Formulation

This section presents the preliminaries and formalizes RUTO.

3.1 Grid Graph

We discretize the study area into a finite Row $\times$ Col undirected grid with node set $V = \{(\text{row}, \text{col}) : 1 \le \text{row} \le \text{Row},\ 1 \le \text{col} \le \text{Col}\}$. Two nodes share an edge if and only if they are 4-neighbors, and each edge has unit hop cost. We denote the resulting edge set by E.

We distinguish between walkable cells (nodes) and plantable cells (nodes). Let $W \subseteq V$ be the set of walkable cells that support pedestrian movement. Let $T \subseteq V$ be the set of plantable cells that are eligible for tree placement. The sets W and T may differ and may overlap; W is constructed by rasterizing pedestrian-network GIS layers, while T is constructed from right-of-way and land-cover layers filtered by exogenous planting rules (e.g., setbacks, utility clearances, and minimum walkway width).

Pedestrian movement occurs on the induced graph $G_{\text{walk}} = (W, E_W)$ with $E_W = \{\{x, y\} \in E : x \in W,\ y \in W\}$. Trees can be placed only on plantable cells, so the decision set always satisfies $S \subseteq T$. By default, planting does not block pedestrian movement, i.e., nodes in $T \cap W$ remain walkable even when selected for planting, so W and G_{walk} are static. This modeling choice simplifies the exposition and keeps shade propagation decoupled from walkability. If planted nodes are treated as non-walkable, we use a decision-dependent walkable set $W(S) = W \setminus S$. In that case the pedestrian graph becomes $G_{\text{walk}}(S)$ and the admissible paths $\mathcal{P}_i$ become $\mathcal{P}_i(S)$. Our analysis remains unchanged because path choice is already decision-dependent in S (agents select $P_i^*(S)$ as a function of S); all definitions extend by replacing W with $W(S)$ where relevant.

3.2 Shading and Benefits

3.2.1 Kernel. A planted tree at $u \in T$ contributes shade to a grid node $v \in V$ through a nonnegative kernel $K(u, v) \in [0, 1]$. We use kernels that depend on the L_1 graph distance $d(u, v)$ on the 4-neighbor grid, i.e., $d(u, v) = |r_u - r_v| + |c_u - c_v|$. In this paper, we consider three canonical forms of K: one hard-cutoff and two soft-decay options. *Hard-cutoff.* $K_{\text{hard}}(u, v; D) = \mathbb{I}[d(u, v) \le D]$ with $D \in \mathbb{N}$. *Exponential soft decay.* $K_{\exp}(u, v; \alpha_{\text{soft}}, R_{\text{soft}}) = \exp(-\alpha_{\text{soft}}\, d(u, v)) \cdot \mathbb{I}[d(u, v) \le R_{\text{soft}}]$ with $\alpha_{\text{soft}} > 0$ and $R_{\text{soft}} \in \mathbb{N}$. *Gaussian soft decay.* $K_{\text{gauss}}(u, v; \ell_{\text{soft}}) = \exp\big(-d(u, v)^2 / (2\ell_{\text{soft}}^2)\big)$ with $\ell_{\text{soft}} > 0$.

3.2.2 Node Shade. Given a tree set $S \subseteq T$, the node shade at v, denoted $h_v(S)$, is the total shade present at node v caused by all trees in S, obtained by summing their kernel contributions $h_v(S) = \sum_{u \in S} K(u, v)$. This quantity is defined on nodes (one scalar per node) and aggregates all trees, in contrast to the contribution from a single tree. With $K \in [0, 1]$ and $|S| \le k$, we have $0 \le h_v(S) \le k$.

3.2.3 Per-node Benefit. We map node shade $h_v(S)$ to a per-node comfort benefit via a concave, nondecreasing transform $\phi : \mathbb{R}_{\ge 0} \to \mathbb{R}_{\ge 0}$ with $\phi(0) = 0$. In words, $\phi(h_v(S))$ is the nonnegative comfort gained when a pedestrian occupies node v under tree set S, defined at the node level and excluding any travel costs. Typical choices of $\phi(\cdot)$ include $\phi_{\text{clip}}(t; \Delta) = \min\{t, \Delta\}$ with $\Delta > 0$, $\phi_{\text{log1p}}(t) = \log(1 + t)$, $\phi_{\text{expsat}}(t; \beta) = 1 - \exp(-\beta t)$ with $\beta > 0$, and $\phi_{\text{power}}(t; \alpha_{\text{power}}) = t^{\alpha_{\text{power}}}$ with $0 < \alpha_{\text{power}} \le 1$.

3.3 Agents and Path Choice

We model each pedestrian trip as an agent characterized by an OD-pair $(s, t) \in W \times W$ sampled from a distribution μ independently of the tree set S.

3.3.1 Paths. A simple path in G_{walk} from s to t is an ordered node sequence $(v_0, \ldots, v_L)$ with $v_0 = s$, $v_L = t$, all v_ℓ distinct, and $\{v_{\ell-1}, v_\ell\} \in E_W$ for each $\ell = 1, \ldots, L$. We write it as $P := \{v_0, \ldots, v_L\}$. For additive expressions we use the node-set notation $v \in P$ with the ordering understood. The admissible path family for (s, t) is denoted by $\mathcal{P}(s, t)$. In computation, we instantiate $\mathcal{P}(s, t)$ as a small fixed set of the shortest simple s–t paths in G_{walk} under the travel-cost metric defined below. It may be further restricted to a corridor-constrained family $\mathcal{P}^{\text{corr}}(s, t; r)$ introduced below.

3.3.2 Corridor Restriction. We optionally restrict admissible routes to an L_1-band corridor around a reference node set. Let $A(s, t) \subseteq W$ summarize plausible centerlines for (s, t) (e.g., $A(s, t) = \text{SP}(s, t)$, where $\text{SP}(s, t)$ denotes the union of nodes on all shortest $s \to t$ paths in G_{walk}). Let $d_{\text{walk}}(x, A) := \min_{y \in A} d_{G_{\text{walk}}}(x, y)$ be hop distance in G_{walk} from node x to a node set A. For integer radius $r \ge 0$, define $C(s, t; r) := \{v \in W : d_{\text{walk}}(v, A(s, t)) \le r\}$ and $\mathcal{P}^{\text{corr}}(s, t; r) := \{P \in \mathcal{P}(s, t) : \{v_0, \ldots, v_L\} \subseteq C(s, t; r)\}$. The width parameter r interpolates from $r = 0$ (exactly $A(s, t)$, e.g., shortest-path union) to no restriction when r exceeds $\max_{v \in W} d_{\text{walk}}(v, A(s, t))$.

3.3.3 Path Choice. Given a tree set S, define the per-path utility $g(S; P) := \sum_{v \in P} \phi(h_v(S)) - \lambda \cdot \text{cst}(P)$, where $\text{cst}(P)$ is a nonnegative travel-cost functional on paths that may depend on hop count, metric length, or other attributes of P (e.g., $\mathrm{cst}(P) = |P| - 1$), and where $\lambda \geq 0$ scales the travel cost. For an OD-pair (s,t), define the best-response path $P^*(S; s, t) := \arg\max^{\mathrm{lex}}_{P \in \mathcal{P}(s,t)} g(S; P)$ (or over $\mathcal{P}^{\mathrm{corr}}(s,t;r)$ if a corridor of width r is enforced), where the superscript "lex" indicates that ties are broken by selecting the lexicographically smallest maximizer.

3.4 Optimization Objective

Using the path utility $g(S; P)$ and the best-response path $P^*(S; s, t)$ defined above, we set

$$\sigma(S) := \mathbb{E}_{(s,t)\sim\mu}\left[g(S; P^*(S; s, t))\right]. \tag{1}$$

The RUTO problem is to maximize $\sigma(S)$ subject to $S \subseteq T$ and $|S| \leq k$, i.e., $S^\circ := \arg\max^{\mathrm{lex}}_{S\subseteq T, |S|\leq k} \sigma(S)$, where S° denotes an optimal planting set.

4 Theoretical Properties

This section states two properties of RUTO.

Theorem 4.1. *The RUTO problem is NP-hard.*

Theorem 4.1 is proved by showing NP-hardness for a restricted special case already contained in the RUTO model family, namely $\lambda = 0$, $\phi(t) = \min\{t, 1\}$, and $K(u,v) = \mathbb{I}[d(u,v) \leq D]$ with $D \geq 1$, with μ supported on finitely many OD-pairs and a corridor of width $r = 0$. Therefore, the general RUTO framework is NP-hard.

We do not claim a complete hardness classification for every further restricted kernel/utility variant. In particular, under some restrictive specifications, the route-switching effect induced by changes in the planting set may disappear, and such variants need not remain NP-hard.

Lemma 4.2. *The objective $\sigma(S)$ is non-submodular.*[1]

Since $\sigma(S)$ is non-submodular, the classic $(1-1/e)$ approximation guarantee does not apply, and without additional assumptions there is no general constant-factor approximation guarantee.

5 Methods

We develop two classes of methods for RUTO, which are theoretically grounded algorithms with solution-dependent approximation guarantees and heuristics with strong empirical performance. We focus on the design of the upper, lower, and proxy objectives.

5.1 Algorithm with Solution-Dependent Approximation Guarantees

Our first method originated from RUTO's close relationship with submodularity. While $\sigma(S)$ is non-submodular, we construct submodular bounds, which we optimize greedily with guarantees by applying the SA strategy.

Algorithm 1 presents the standard greedy selection procedure, which forms the basis of our optimization algorithms. Starting from the empty set, it repeatedly adds the plantable node with the largest marginal gain until the budget k is exhausted.

Algorithm 1: GreedySelection

Input: Plantable set T; objective function $F(\cdot)$; budget k
Output: Selected tree set S_{greedy}

```
Initialize S ← ∅
for i ← 1, 2, ..., k do
    Choose v* ∈ T \ S that maximizes the marginal gain
      F(S ∪ {v}) − F(S) (ties broken lexicographically)
    Update S ← S ∪ {v*}
return S as S_greedy
```

Algorithm 2: SandwichApproximation

Input: Plantable set T; budget k; monotone submodular upper bound $\mathrm{UB}(\cdot)$; monotone submodular lower bound $\mathrm{LB}(\cdot)$; original objective $\sigma(\cdot)$
Output: Sandwich solution S_{sand}

```
S_ub ← GreedySelection(T, UB, k) using Alg. 1
S_lb ← GreedySelection(T, LB, k) using Alg. 1
Estimate σ(S_ub) and σ(S_lb) by Monte Carlo sampling over
  OD-pairs
S_sand ← argmax_{S ∈ {S_ub, S_lb}} σ(S)
return S_sand
```

5.1.1 Sandwich Approximation Strategy. The SA strategy addresses non-submodular maximization by optimizing monotone submodular bounds that bracket the objective [22]. Algorithm 2 summarizes its procedure. It runs greedy separately on the upper and lower bounds, evaluates the two resulting solutions under the original objective $\sigma(\cdot)$, and returns the better one.

Assume we have LB and UB that are monotone and submodular and satisfy $\mathrm{LB}(S) \leq \sigma(S) \leq \mathrm{UB}(S)$ for all $S \subseteq T$. Let S_{ub} and S_{lb} denote the solutions returned by the greedy algorithm maximizing UB and LB, respectively. Define S_{sand} as the better of these two candidates when evaluated by σ. Then, we have

$$\sigma(S_{\mathrm{sand}}) \geq \max\left\{\frac{\sigma(S_{\mathrm{ub}}) - \mathrm{UB}(\varnothing)}{\mathrm{UB}(S_{\mathrm{ub}}) - \mathrm{UB}(\varnothing)}, \frac{\mathrm{LB}(S^\circ) - \mathrm{LB}(\varnothing)}{\sigma(S^\circ) - \mathrm{UB}(\varnothing)}\right\} \cdot (1 - 1/e)\left(\sigma(S^\circ) - \mathrm{UB}(\varnothing)\right) + \mathrm{LB}(\varnothing). \tag{2}$$

Note that Eq. (2) differs from the corresponding inequality (Eq. (4)) in [22] because we do not require the bounds to vanish at the empty set (i.e., $\mathrm{UB}(\varnothing)$ and $\mathrm{LB}(\varnothing)$ may be nonzero). The factor $(1 - 1/e)$ is the approximation ratio on the normalized submodular objectives. The two fractions inside the max operator quantify bound tightness after normalization, and the outer addition of $\mathrm{LB}(\varnothing)$ shifts the guarantee back to the original scale. When $\mathrm{UB}(\varnothing) = \mathrm{LB}(\varnothing) = 0$ Eq. (2) reduces to the standard SA inequality. A complete proof of Eq. (2) is provided in the appendix.

5.1.2 Upper Bounds. The first upper bound is based on a simple observation. For each OD-pair, the value on any admissible path is no larger than the value obtainable on the union of all admissible nodes. Likewise, the travel cost on any admissible path is at least the minimal admissible cost $C_{\min}(s,t) := \min_{P\in\mathcal{P}(s,t)} \mathrm{cst}(P)$.

[1] Let U be a finite ground set and $f : 2^U \to \mathbb{R}$. We say f is submodular if and only if for all $A \subseteq B \subseteq U$ and $x \in U \setminus B$, $f(A \cup \{x\}) - f(A) \geq f(B \cup \{x\}) - f(B)$.

If $\mathrm{cst}(P) = |P| - 1$, then $C_{\min}(s,t)$ coincides with the cost of a shortest admissible path, i.e., the derivation does not depend on a specific cost choice. We define $U(s,t) \coloneqq \bigcup_{P \in \mathcal{P}(s,t)} P$, i.e., the union of nodes on all admissible paths in $\mathcal{P}(s,t)$. Then, we obtain the following upper bound:

$$\mathrm{UB}_1(S) \coloneqq \mathbb{E}_{(s,t)\sim\mu}\left[\sum_{v \in U(s,t)} \phi(h_v(S)) - \lambda C_{\min}(s,t)\right]. \tag{3}$$

The second upper bound is based on the following intuition. Per node shade is at most k since $|S| \le k$ and $0 \le K(u,v) \le 1$. Hence the benefit along any admissible path is at most its length times $\phi(k)$. This yields a per OD cap that is independent of S, and combining it with the union-of-paths bound gives a tighter expression, i.e.,

$$\mathrm{UB}_2(S) \coloneqq \mathbb{E}_{(s,t)\sim\mu}\left[\min\left\{B(s,t), \sum_{v \in U(s,t)} \phi(h_v(S))\right\} - \lambda C_{\min}(s,t)\right], \tag{4}$$

where $B(s,t) \coloneqq \max_{P \in \mathcal{P}} |P| \cdot \phi(k)$ denotes a per OD cap on attainable path benefit, i.e., the path length upper bound times the per node maximum benefit under at most k trees.

The following two lemmas establish submodularity and validity of the two upper bounds, as well as their ordering.

Lemma 5.1. *Both $\mathrm{UB}_1(S)$ and $\mathrm{UB}_2(S)$ are submodular in S.*

Lemma 5.2. *For any tree set $S \subseteq T$, $\sigma(S) \le \mathrm{UB}_2(S) \le \mathrm{UB}_1(S)$.*

5.1.3 Lower Bounds. The first lower bound uses a path-level surrogate, i.e., for each OD-pair we evaluate a prescribed admissible path instead of optimizing over $\mathcal{P}(s,t)$. Let $P^{\mathrm{sel}}(s,t) \in \mathcal{P}(s,t)$ be any measurable choice, e.g., a shortest path or a corridor centerline. We then define

$$\mathrm{LB}_1(S) = \mathbb{E}_{(s,t)\sim\mu}\left[g(S; P^{\mathrm{sel}}(s,t))\right]. \tag{5}$$

The second lower bound averages over a fixed family of admissible paths, i.e., we use a distribution $q_{s,t}$ on $\mathcal{P}(s,t)$ that does not depend on S. Examples include a uniform mixture over K-shortest paths or corridor samples.

$$\mathrm{LB}_2(S) = \mathbb{E}_{(s,t)\sim\mu}\left[\mathbb{E}_{P\sim q_{s,t}}\left[g(S;P)\right]\right]. \tag{6}$$

The third lower bound uses a data-adaptive mixture that approximates best responses while avoiding the explicit max over $\mathcal{P}(s,t)$. Define the softmax distribution $q_{s,t}^{(S)}$ on $\mathcal{P}(s,t)$ with temperature $\tau > 0$ by $q_{s,t}^{(S)}(P) = \frac{\exp(\tau \cdot g(S;P))}{\sum_{P \in \mathcal{P}(s,t)} \exp(\tau \cdot g(S;P))}$. The dependence on S is explicit both in the superscript and through $g(S;\cdot)$. For any fixed $S_i \subseteq V$ $(i \in \mathbb{N})$, we define the third lower bound

$$\mathrm{LB}_3^{(i)}(S) = \mathbb{E}_{(s,t)\sim\mu}\left[\mathbb{E}_{P\sim q_{s,t}^{(S_i)}}\left[g(S;P)\right]\right]. \tag{7}$$

We use a recursive scheme that freezes the mixture at the current iterate and maximizes the frozen bound with respect to S (see Alg. 3). Formally, $S_{i+1} = \arg\max^{\mathrm{lex}}_{S \subseteq T,\ |S| \le k} \mathrm{LB}_3^{(i)}(S)$, with S_0 initialized by any tree set, e.g., the output of another method. By construction we have $\mathrm{LB}_3^{(i)}(S_{i+1}) \ge \mathrm{LB}_3^{(i)}(S_i)$. After each update we refit the mixture $q_{s,t}^{(S_{i+1})}$ for the next step. The refitting preserves validity, i.e., $\mathrm{LB}_3^{(i+1)}(S_{i+1}) \le \sigma(S_{i+1})$, but it does not guarantee monotonic growth across iterations. In practice, we adopt the largest frozen value $\mathrm{LB}_3^{(i)}(S_{i+1})$ across all steps.

Algorithm 3: Recursive Computation of LB_3

Input: Plantable set T; budget k; initializer S_0; max iterations I; tolerance γ

```
for i ← 0, 1, ..., I − 1 do
    S_{i+1} ← arg max^{lex}_{S⊆T, |S|≤k} LB_3^{(i)}(S);
    if LB_3^{(i)}(S_{i+1}) − LB_3^{(i)}(S_i) ≤ γ then
        Break
return S_{i+1}
```

The following two lemmas establish the submodularity and validity of the three lower bounds.

Lemma 5.3. *Each of $\mathrm{LB}_1(S)$, $\mathrm{LB}_2(S)$, and $\mathrm{LB}_3^{(i)}(S)$ is submodular for any fixed i.*

Lemma 5.4. *For any $S \subseteq T$, $\mathrm{LB}_1(S) \le \sigma(S)$, $\mathrm{LB}_2(S) \le \sigma(S)$, and $\mathrm{LB}_3^{(i)}(S) \le \sigma(S)$ for any fixed i.*

5.2 Effective Heuristic Algorithms

We introduce two proxy objectives in this section. They guide the choice of S without theoretical guarantees. For each OD-pair choose a proxy family $\widehat{\mathcal{P}}(s,t) \subseteq \mathcal{P}(s,t)$ such as K-shortest paths or corridor samples. Similar to the definition of $U(s,t)$ in Sec. 5.1.2, let $\widehat{U}(s,t) \coloneqq \bigcup_{P \in \widehat{\mathcal{P}}(s,t)} P$. Define the total multiplicity mass $M(s,t) \coloneqq \sum_{P \in \widehat{\mathcal{P}}(s,t)} |P|$ and the retention ratio $\rho_1(s,t) \coloneqq \frac{|\widehat{U}(s,t)|}{M(s,t)}$. The first proxy objective is defined as

$$\begin{aligned}\mathrm{PRX}_1(S) &\coloneqq \mathbb{E}_{(s,t)\sim\mu}\Bigg[\frac{1}{\rho_1(s,t)|\widehat{\mathcal{P}}(s,t)|} \cdot \\ &\left(\left(\sum_{v \in \widehat{U}(s,t)} \phi(h_v(S))\right) - \lambda\, \rho_1(s,t) \cdot \left(\sum_{P \in \widehat{\mathcal{P}}(s,t)} \mathrm{cst}(P)\right)\right)\Bigg].\end{aligned} \tag{8}$$

The rationale behind Eq. (8) is to deduplicate benefits across proxy paths while keeping cost and normalization on the same basis. PRX_1 collapses $\widehat{\mathcal{P}}(s,t)$ to the union $\widehat{U}(s,t)$ so each node contributes at most once, i.e., it avoids inflating scores when many combinatorial variants traverse the same nodes. The retention ratio ρ_1 measures the fraction of total node visits that remain after de-duplication. We scale both the normalization and the cost by ρ_1 so that benefit and cost are matched to the same level of counting. This prevents corridors with many near-duplicate routes from dominating the score. When proxy paths are largely disjoint we have $\rho_1 \approx 1$ and PRX_1 behaves like a simple average over paths. When proxy paths strongly overlap we have $\rho_1 \approx 0$ and PRX_1 discounts both benefit and cost consistently rather than only the benefit. All path-set dependent terms are precomputable, while $\phi(h_v(S))$ updates with S, so PRX supports fast greedy selection under $|S| \le k$.

We generalize PRX_1 by allowing at most η path hits per node and rescaling by the corresponding retention ratio. Define the capped multiplicity mass $M_\eta(s,t) \coloneqq \sum_{v \in \widehat{U}(s,t)} \min\left\{\eta, \sum_{P \in \widehat{\mathcal{P}}(s,t)} \mathbb{I}[v \in P]\right\}$.

Define $\rho_\eta(s,t) \coloneqq M_\eta(s,t)/M(s,t) \in (0,1]$. Then, we set

$$\mathrm{PRX}_\eta(S) \coloneqq \mathbb{E}_{(s,t)\sim\mu}\Bigg[\frac{1}{\rho_\eta(s,t)|\widehat{\mathcal{P}}(s,t)|}\cdot\Bigg(\Bigg(\sum_{v\in\widehat{U}(s,t)}\phi(h_v(S))\cdot \min\Bigg\{\eta,\sum_{P\in\widehat{\mathcal{P}}(s,t)}\mathbb{I}[v\in P]\Bigg\}\Bigg)-\lambda\,\rho_\eta(s,t)\Bigg(\sum_{P\in\widehat{\mathcal{P}}(s,t)}\mathrm{cst}(P)\Bigg)\Bigg)\Bigg]. \tag{9}$$

The rationale behind Eq. (9) is analogous to Eq. (8), i.e., we cap per node path hits at η and rescale both the normalization and the cost by $\rho_\eta(s,t)$ so that benefit and cost use the same counting base. Eq. (8) is exactly the $\eta = 1$ case of Eq. (9), and the two expressions coincide, so we do not distinguish them when $\eta = 1$.

5.3 Sampling

All objectives in Eqs. (3)–(9) are written as expectations over $(s,t) \sim \mu$. Computing these expectations exactly is intractable because the distribution μ ranges over up to $O(|W|^2)$ OD-pairs and, for each pair, the agent utility requires a maximization over an exponentially large set of s-to-t paths. Thus, we estimate these expectations by drawing i.i.d. OD-pairs from μ and set the sample size via Hoeffding's inequality [13]. For any $\varepsilon > 0$ and $\delta \in (0,1)$, this yields an (ε,δ)-approximation. We refer the reader to the appendix for a more detailed summary.

5.4 The Overall Algorithmic Process

Algorithm 4 presents the overall procedure for solving RUTO. The process has three stages. First, we sample m i.i.d. OD-pairs from μ and construct the admissible-path information required by the selected objective. Second, we replace the expectation in the objective with an empirical average over the sampled OD-pairs, obtaining an empirical objective (or empirical upper/lower bounds). Third, we optimize this empirical objective either by direct greedy selection or, when using the SA framework, by applying the sandwich-approximation routine to the empirical bounds. This provides a unified implementation view for all objectives in Eqs. (3)–(9).

5.5 Complexity

We evaluate all methods with m i.i.d. OD-pair samples. Let κ be the kernel support per tree, i.e., $\kappa = |\{v \in V : K(u,v) > 0\}|$. Let c_{route} denote the per-OD routing cost to construct the admissible path family required by the chosen objective, e.g., computing a shortest-path union or a corridor band via BFS on G_{walk}. The total time cost is $O(m \cdot c_{\mathrm{route}} + k \cdot \kappa \cdot |T|)$ and the space cost is $O(|V| + \kappa \cdot |T|)$. To express the complexity in terms of the accuracy parameters (ε,δ), we apply Hoeffding's inequality. Setting m with target (ε,δ) gives $m \ge \frac{(b-a)^2}{2\varepsilon^2}\log\frac{2}{\delta}$, where $[a,b]$ is the range for the per-OD contribution $Y(S;(s,t))$ of the chosen objective, i.e., $a \le Y(S;(s,t)) \le b$ for all (s,t). Substituting this yields total time $O\big(\frac{(b-a)^2}{\varepsilon^2}\log\frac{2}{\delta}\cdot c_{\mathrm{route}} + k\cdot\kappa\cdot|T|\big)$, with the same space bound.

6 Experiments

We evaluate the proposed algorithms against several baselines while varying hyperparameters. All experiments are run on a Linux server. The code is implemented in C++17, compiled with the -O2 optimization flag.

Algorithm 4: Overall RUTO Solver

Input: OD distribution μ (or OD dataset); plantable set T; budget k; sample size m; choice of objective type (upper, lower, or proxy); optional use of SA strategy (Alg. 2)
Output: Tree node set S

```
Sample m i.i.d. OD-pairs (s_i, t_i) ~ μ for i = 1, ..., m
For each (s_i, t_i), construct the admissible path information
  required by the chosen objective (e.g., corridor family,
  proxy paths, or softmax mixture statistics)
Construct the empirical objective F̂(·) by replacing
  expectations in the chosen objective with averages over
  {(s_i, t_i)}_{i=1}^m
if using a single objective (upper, lower, or proxy) then
    S ← GreedySelection(T, F̂, k) using Alg. 1
else if using the SA strategy then
    Instantiate empirical upper and lower bounds UB̂(·) and
      LB̂(·) on the sample
    S ← SandwichApproximation(T, k, UB̂, LB̂, σ) using
      Alg. 2
return S
```

6.1 Experimental Settings

This section introduces the parameter settings and the compared algorithms.

Parameters. By default, we use a 64×64 2D-grid with budget $k = 100$, and all nodes are walkable and plantable. By default, agents' OD-pairs are sampled i.i.d. from a distribution μ that samples origins and destinations independently and uniformly over the node set. Per-path utility g is defined with $\lambda = 0.5$, and the travel cost of a path is its length. The kernel K is K_{hard} with cutoff $D = 2$ (Section 3.2.1), and the node-benefit transform ϕ is ϕ_{clip} with $C = 2.0$ (Section 3.2.3). For each agent, we enumerate at most ten shortest paths as candidates, without corridor constraints, and break ties lexicographically. LB_2 adopts the uniform distribution $q_i^{(0)}$ over the enumerated candidate paths (see Eq. (6)). LB_3 adopts a softmax with temperature $\tau = 1.0$ and is run for at most $I = 5$ iterations with a tolerance of 10^{-6} (see Eq. (7) and Algorithm 3). For node selection, each run samples $m = 10^5$ agents, and for evaluation each run independently samples another 10^5 agents. Notably, we do not set m via an (ε,δ) criterion and instead fix m for convenience. To assess sensitivity, we also treat m as a tunable hyperparameter and run an additional group of experiments. Beyond the default choices for the OD-pair distribution μ, the kernel K, and the concave transform ϕ, we also evaluate several variants. Details are given in Section 6.2. Each experiment is repeated five times, and results are averaged to mitigate randomness.

Compared algorithms. We compare thirteen methods, including six SA-based algorithms with theoretical guarantees, four heuristics, and three naive baselines. The SA-based algorithms combine the two upper bounds with the three lower bounds in Section 5.1, i.e., all pairings $\mathrm{UB}_x + \mathrm{LB}_y$ with $x \in \{1,2\}$ and $y \in \{1,2,3\}$. We denote each instance by *SA* (UB_x+LB_y), e.g., *SA* (UB_1+LB_1). The heuristics in Section 5.2 use the objective in Eq. (9), and we denote the variant with parameter η by $PROXY_\eta$ for $\eta \in \{1, 2, 4, 8\}$. As a baseline, we use a greedy method (denoted *Greedy*) that iteratively adds to S the node that maximizes the current utility and then updates marginal gains lazily, recomputing only for agents whose best-response utility can change due to the newly added node. We use a deterministic regular-spacing baseline (denoted *Regular*) that places k trees so that the inter-tree spacing is as uniform as the grid allows. We also use a baseline (denoted *Random*) that selects k nodes uniformly at random from T.

6.2 Experimental Results

We organize the experiments into several groups. For each group, we vary one hyperparameter and report three metrics: the objective value σ, the runtime, and the approximation factor. In Eq. (2), the approximation factor is $(1 - 1/e)$ times the maximum of two arguments. The second argument, $\frac{\mathrm{LB}(S^\circ)-\mathrm{LB}(\varnothing)}{\sigma(S^\circ)-\mathrm{UB}(\varnothing)}$, depends on the unknown optimal set S° and is thus not computable. We therefore report the first argument, $\frac{\sigma(S_{\mathrm{ub}})-\mathrm{UB}(\varnothing)}{\mathrm{UB}(S_{\mathrm{ub}})-\mathrm{UB}(\varnothing)}$, as an auxiliary quantity that provides a valid lower bound on the maximum.

6.2.1 Varying budget k under different distributions μ. The first group of experiments evaluates how the algorithms perform as the budget k varies over $\{50, 100, 150, 200\}$. This group is our primary evaluation, thus in addition to the default uniform setting, we consider three non-uniform OD-pair distributions μ that better capture realistic pedestrian preferences.

First, the distance-window distribution μ_{win} samples OD-pairs uniformly and keeps only those whose shortest-path length lies within a prescribed range b ($b = 32$ by default), using rejection sampling until m pairs are accepted. Second, the hotspot-weighted distribution μ_{hot} assigns each node a weight and samples source and destination nodes with probability proportional to these weights, concentrating demand around busy locations. The instance of μ_{hot} on the 2D-grid used in our experiments is shown in the appendix. Third, the gravity-decayed distribution μ_{grav} proposes OD-pairs uniformly and accepts each with probability $\exp(-\beta_{\mathrm{gravity}} d(s,t))$, where $d(s,t)$ denotes its shortest-path length. We set $\beta_{\mathrm{gravity}} = 0.1$ by default and continue sampling until m OD-pairs are retained.

The results are shown in Figure 2. Across all settings, the six SA-based algorithms and the four heuristic methods achieve objective values that are comparable to *Greedy*. They also clearly outperform the simple baselines *Regular* and *Random*, which indicates that exploiting our objectives yields a substantial gain over naive strategies. In terms of efficiency, our methods evaluate marginal gains without recomputing the objective after each planted node. As a consequence, they are roughly an order of magnitude faster than *Greedy*. The *Random* and *Regular* baselines have essentially short running time, so we omit them from the runtime figures. Among our methods, the proxy heuristics are the fastest, whereas the SA-based variants incur moderate additional overhead due to maintaining upper and lower bounds. For auxiliary quantity $\frac{\sigma(S_{\mathrm{ub}})-\mathrm{UB}(\varnothing)}{\mathrm{UB}(S_{\mathrm{ub}})-\mathrm{UB}(\varnothing)}$, across all six SA-based algorithms, it remains at a constant level in all experiments, in particular it is always larger than 0.2. Moreover, UB_2-based algorithms achieve larger auxiliary quantities than those based on UB_1, which is consistent with Lemma 5.2 since UB_2 is tighter. These observations suggest that the solution-dependent guarantees provided by our upper/lower bounds are practically meaningful as well as theoretically sound. Overall, among the SA-based variants, $\mathrm{UB}_2 + \mathrm{LB}_2$ is the most natural default choice in our experiments, since UB_2 is tighter than UB_1, LB_2 is generally preferable to LB_1, and LB_3 is more expensive without consistently improving over LB_2.

6.2.2 Varying sample size m. We vary the sample size m over $\{10^3, 5\times10^3, \ldots, 10^6\}$. The results are shown in Figure 3. The objective is nearly unchanged across different m, while its standard error decreases as m increases. Larger m also leads to longer runtime. Overall, our default choice $m = 10^5$ strikes a balance between accuracy and runtime cost without materially affecting the comparison among the algorithms.

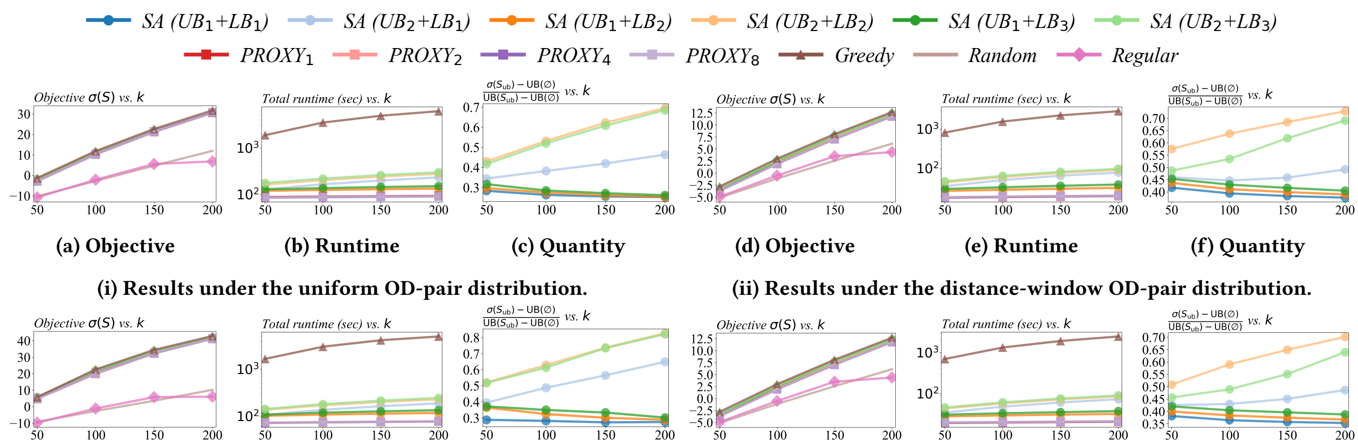
6.2.3 Varying parameters in LB_3. We study the sensitivity of the lower bound LB_3 to its hyperparameters. First, we fix the softmax temperature $\tau = 1.0$ and vary the iteration cap $I \in \{3, 5, 10, 20\}$. Second, we fix $I = 5$ and vary $\tau \in \{0.5, 1.0, 1.5, 2.0\}$. Table 2 in the appendix reports our results. All tested parameter combinations yield very similar objectives. Varying τ has virtually no effect on runtime, whereas increasing I leads to longer runtime, but still within the same order of magnitude. Thus, our default setting ($\tau = 1.0$, $I = 5$) is reasonable and representative of the behavior of LB_3.

6.2.4 Varying grid size. We vary the grid size (number of nodes) across 16×16, 64×64, 256×256, and 1024×1024. We skip *Greedy* on the 1024×1024 grid because its runtime is prohibitive. Figure 4 reports the runtime and auxiliary quantity results. As expected, the runtime increases with grid size. The reported quantity decreases as the grid grows, since a larger grid induces more nodes in the union-of-paths set (see Eq. (3) and Eq. (4)). However, its value remains at a constant level across all grid sizes. Our SA-based algorithms match *Greedy* in objective value across all grid sizes. The objective curves for these methods, which increase with grid size, are omitted from Figure 4 due to space limits.

6.2.5 Varying travel cost parameter λ. We study the effect of λ on algorithmic performance. As shown in Figure 5, our proposed methods achieve essentially the same objective values as *Greedy* across all tested values of λ. Runtime and approximation auxiliary quantities do not vary with λ, so we omit their plots for brevity.

6.2.6 Varying Kernel function $K(\cdot)$. As introduced in Section 3.2.1, we evaluate different shading kernels. We consider $K_{\mathrm{hard}}(u, v; D)$ with varying D, $K_{\exp}(u, v; \alpha_{\mathrm{soft}}, R_{\mathrm{soft}})$ with varying α_{soft} ($R_{\mathrm{soft}} = 12$), and $K_{\mathrm{gauss}}(u, v; \ell_{\mathrm{soft}})$ with varying ℓ_{soft}. We use the transform ϕ_{log1p}. Our methods are substantially faster than *Greedy* while attaining essentially identical objective values (curves are deferred to the appendix for brevity). Interestingly, for the UB_2-based algorithms, the approximation auxiliary quantity first decreases slightly and then increases as the effective shading range of the kernel grows.

6.2.7 Varying transformation function $\phi(\cdot)$. As introduced in Section 3.2.3, we consider multiple forms of the transform ϕ. We vary the parameter Δ of ϕ_{clip}, β of ϕ_{expsat}, and α_{power} of ϕ_{power}. We defer these plots to the full version, as they are similar to those above and show no new trends.



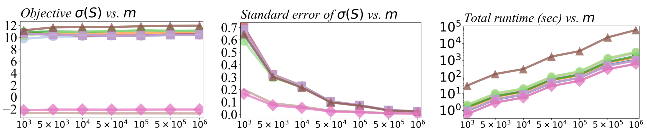
(a) Objective **(b) Runtime** **(c) Quantity** **(d) Objective** **(e) Runtime** **(f) Quantity**

(i) Results under the uniform OD-pair distribution. **(ii) Results under the distance-window OD-pair distribution.**

(g) Objective **(h) Runtime** **(i) Quantity** **(j) Objective** **(k) Runtime** **(l) Quantity**

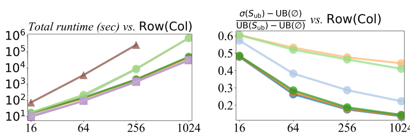
(iii) Results under the hotspot-weighted OD-pair distribution. **(iv) Results under the gravity-decayed OD-pair distribution.**

Figure 2: Objective, runtime, and auxiliary quantity as the budget k varies under four OD-pair distributions.



(a) Objective **(b) Standard error** **(c) Runtime**

Figure 3: Objective, standard error of the objective, and runtime as the sample size m varies.



(a) Runtime **(b) Quantity**

Figure 4: Runtime and approximation auxiliary quantity as the grid size varies.

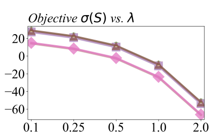


Figure 5: Objective $\sigma(S)$ as the travel cost λ varies.

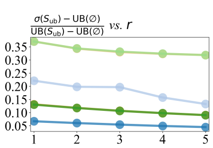
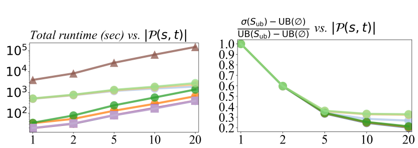


Figure 6: Approximation auxiliary quantity as corridor width r varies.



(a) Runtime **(b) Quantity**

Figure 7: Runtime and approximation auxiliary quantity as $|\mathcal{P}(s,t)|$ varies.

6.2.8 Varying corridor width r. The corridor width r (introduced in Section 3.3.2) does not materially change the effectiveness or efficiency of our methods relative to the baselines. As shown in Figure 6, the reported quantity decreases slightly as r grows, which is consistent with intuition.

6.2.9 Varying admissible path family size $|\mathcal{P}(s,t)|$. We vary the number of admissible paths per OD-pair (see Section 3.3.1). The results are shown in Figure 7. Larger $|\mathcal{P}(s,t)|$ naturally increases the runtime for all methods, but the growth for our proposed algorithms is relatively moderate compared with *Greedy*. Larger $|\mathcal{P}(s,t)|$ also leads to a smaller reported quantity, and when $|\mathcal{P}(s,t)| = 1$, we have $\frac{\sigma(S_{\mathrm{ub}})-\mathrm{UB}(\emptyset)}{\mathrm{UB}(S_{\mathrm{ub}})-\mathrm{UB}(\emptyset)} = 1$, which is consistent with our definition.

6.2.10 Varying parameters of different distributions. Recall that in Section 6.2.1 we introduced three distributions, namely μ_{win}, μ_{hot}, and μ_{grav}. For μ_{win}, we vary its hyperparameter $b \in \{8, 16, 32, 64\}$. For μ_{grav}, we vary its hyperparameter $\beta_{\mathrm{gravity}} \in \{0.05, 0.1, 0.2, 0.4\}$. For μ_{hot}, we consider four hotspot patterns, *center*, *axis*, *triangle*, and *periphery*, which contain 1, 2, 3, and 4 hotspot regions, respectively. Heatmap visualizations of these patterns are provided in the full version of the paper. Results are consistent with all previous experiments.

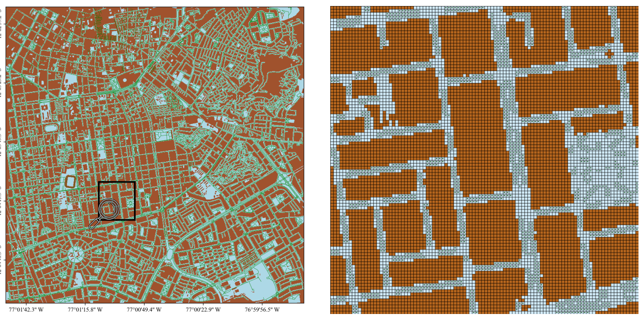
6.2.11 Varying plantable set T and walkable set W. We consider four geometrically structured layouts for the plantable set T and four for the walkable set W, using regular patterns on the grid. We vary one of T or W while keeping the other at its default shape. The relative performance remains consistent with previous experiments. Details are deferred to the full version.

6.2.12 Scalability Limits of Exact and Meta-Heuristic Baselines. We also compare our method with three additional baselines on small grids under the default setting: *Exact* (exhaustive search under the full bilevel objective), *Exact-Sampled (ES)* (exhaustive search under the sampled objective), and *SimAnneal* (simulated annealing on the sampled objective). For all sampled-objective methods in this comparison, we use 10^5 sampled agents, consistent with *Greedy*.

Table 1 reports the results on a (5×5) grid. All four methods achieve very similar objective values, while our SA-based method

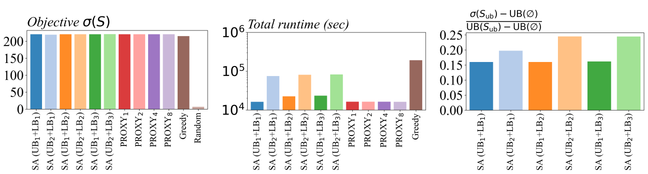
Table 1: Objective and runtime comparison with exact and meta-heuristic baselines on a (5×5) grid.

	Exact	*ES*	*SimAnneal*	*SA(UB1+LB1)*
Objective	6.427	6.424	6.425	6.365
Time (s)	95.0	32051	4315	5.62



(a) Rasterized pedestrian network in Lima. **(b) A local zoom of the pedestrian network (corresponding to the black square in panel (a)).**

Figure 8: Pedestrian network in Lima with a local zoom.



(a) Objective **(b) Runtime** **(c) Quantity**

Figure 9: Objective, runtime, and approximation auxiliary quantity in the real-world case study on Lima, Peru.

is substantially faster than the additional baselines. Although this runtime ordering may appear counterintuitive, it is reasonable for such a tiny grid. *Exact* remains cheap at this scale, whereas *ES* already incurs the fixed Monte Carlo cost with 10^5 agents. On slightly larger grids, e.g., (8×8), these exact and meta-heuristic baselines quickly become impractical. This also highlights the main advantage of our approach. Generic meta-heuristics still suffer from repeated simulations caused by the bilevel structure, whereas our bound-based method is designed to reduce this evaluation cost.

6.3 Empirical Studies

We also evaluate our methods on real-world pedestrian networks. We extract OpenStreetMap (OSM) data for four large tropical cities with high population density and relatively low tree cover, namely Lima (Peru), Dhaka (Bangladesh), Jakarta (Indonesia), and Cairo (Egypt). For each city, we select a central location, take a 4 km×4 km region around it, and rasterize this region into 5 m × 5 m grid cells.

Taking Lima as an example, Figure 8a shows the rasterized city patch, where brown regions indicate buildings, green lines are OSM pedestrian routes, and blue cells represent walkable areas. We define the plantable set T as grid cells whose centers are sufficiently far from buildings and close to pedestrian routes and intersections, and the walkable set W consists of outdoor areas in OSM. Figure 8b shows a local zoom around the Gamarra area[2], where green dots mark plantable cells in T. For OD sampling, we restrict candidate origins and destinations to walkable nodes adjacent to buildings and assign them unit weight, so that agents enter and exit the network only near building frontages rather than in the middle of wide roads. Further details of the city preprocessing and parameters are deferred to the full version of the paper.

On the resulting Lima grid, which has 650,436 nodes, we set the budget to $k = 20{,}000$, a reasonable scale for dense street-tree planning. Since roughly 18% of plantable cells are selected, *Random* mimics a naive policy that plants trees along many streets without optimization. The *Regular* baseline is not included, because the plantable set T is highly irregular and approximate equal spacing is difficult to enforce. For *Greedy* we use a smaller sample size $m = 10^4$ due to its high cost, while all other methods use $m = 10^5$.

The results are shown in Figure 9. The relative performance of the methods is consistent with the synthetic-grid experiments. These results indicate that RUTO can be optimized effectively and efficiently on realistic city-scale networks by our algorithms.

7 Discussion and Future Work

Our current study is limited by the lack of rich real-world data, e.g., fine-grained pedestrian heatmaps and empirically validated models of thermal comfort. Consequently, our settings are necessarily stylized. One important direction for future work is to calibrate RUTO using empirical pedestrian flows and microclimate measurements. Another is to incorporate stochastic demand prediction and route-choice modeling, so that the framework can better capture heterogeneous and uncertain pedestrian behavior.

On the positive side, our framework is highly scalable and not specific to street trees. More broadly, our bilevel paradigm could be adapted to a variety of urban intervention siting problems in which facility placement affects pedestrians' route utility and hence their route choices, e.g., locating public restrooms or WiFi hotspots.

8 Conclusion

We formalize RUTO, which couples urban tree placement with route choices on pedestrian networks. Despite its bilevel and non-submodular nature, we exploit its max-cover flavor by constructing submodular upper and lower bounds and applying the SA strategy with solution-dependent guarantees. We further propose proxy objectives for faster heuristics. Experiments on synthetic grids and real-world pedestrian networks demonstrate that our methods achieve strong solution quality with substantially reduced runtime.

Acknowledgments

This work was partially supported by JSPS KAKENHI Grant Number JP22H00533, Japan.

[2]Gamarra area is the largest textile commercial district in South America.

References

[1] Sara Ahmadian, Babak Behsaz, Zachary Friggstad, Amin Jorati, Mohammad R. Salavatipour, and Chaitanya Swamy. 2018. Approximation Algorithms for Minimum-Load k-Facility Location. *ACM Trans. Algorithms* 14, 2, Article 16 (April 2018), 29 pages. https://doi.org/10.1145/3173047

[2] Sara Ahmadian, Zachary Friggstad, and Chaitanya Swamy. 2013. Local-search based approximation algorithms for mobile facility location problems. In *Proceedings of the Twenty-Fourth Annual ACM-SIAM Symposium on Discrete Algorithms* (New Orleans, Louisiana) *(SODA '13)*. Society for Industrial and Applied Mathematics, USA, 1607–1621.

[3] Omar Ben-Ayed, David E. Boyce, and Charles E. Blair. 1988. A general bilevel linear programming formulation of the network design problem. *Transportation Research Part B: Methodological* 22, 4 (1988), 311–318. https://doi.org/10.1016/0191-2615(88)90006-9

[4] Martha-Selene Casas-Ramírez, José-Fernando Camacho-Vallejo, Juan A Díaz, and Dolores E Luna. 2020. A bi-level maximal covering location problem. *Operational Research* 20, 2 (2020), 827–855. https://doi.org/10.1007/s12351-017-0357-y

[5] Richard Church and Charles R Velle. 1974. The maximal covering location problem. *Papers in Regional Science* 32, 1 (1974), 101–118. https://doi.org/10.1111/j.1435-5597.1974.tb00902.x

[6] Bianca Milani de Quadros and Martin Gabriel Ordenes Mizgier. 2023. Urban green infrastructures to improve pedestrian thermal comfort: A systematic review. *Urban Forestry & Urban Greening* 88 (2023), 128091. https://doi.org/10.1016/j.ufug.2023.128091

[7] Juan A. Díaz, Dolores E. Luna, José-Fernando Camacho-Vallejo, and Martha-Selene Casas-Ramírez. 2017. GRASP and hybrid GRASP-Tabu heuristics to solve a maximal covering location problem with customer preference ordering. *Expert Systems with Applications* 82 (2017), 67–76. https://doi.org/10.1016/j.eswa.2017.04.002

[8] Ian Estacio, Rafik Hadfi, Ariel Blanco, Takayuki Ito, and Jennieveive Babaan. 2022. Optimization of tree positioning to maximize walking in urban outdoor spaces: A modeling and simulation framework. *Sustainable Cities and Society* 86 (2022), 104105. https://doi.org/10.1016/j.scs.2022.104105

[9] Ziyou Gao, Jianjun Wu, and Huijun Sun. 2005. Solution algorithm for the bi-level discrete network design problem. *Transportation Research Part B: Methodological* 39, 6 (2005), 479–495. https://doi.org/10.1016/j.trb.2004.06.004

[10] Aristides Gionis, Theodoros Lappas, Konstantinos Pelechrinis, and Evimaria Terzi. 2014. Customized tour recommendations in urban areas. In *Proceedings of the 7th ACM International Conference on Web Search and Data Mining* (New York, New York, USA) *(WSDM '14)*. Association for Computing Machinery, New York, NY, USA, 313–322. https://doi.org/10.1145/2556195.2559893

[11] Bruce L Golden, Larry Levy, and Rakesh Vohra. 1987. The orienteering problem. *Naval Research Logistics (NRL)* 34, 3 (1987), 307–318. https://doi.org/10.1002/1520-6750(198706)34:3<307::AID-NAV3220340302>3.0.CO;2-D

[12] Tongping Hao, Qunshan Zhao, and Jianxiang Huang. 2023. Optimization of tree locations to reduce human heat stress in an urban park. *Urban Forestry & Urban Greening* 86 (2023), 128017. https://doi.org/10.1016/j.ufug.2023.128017

[13] Wassily Hoeffding. 1963. Probability Inequalities for Sums of Bounded Random Variables. *J. Amer. Statist. Assoc.* 58, 301 (1963), 13–30. https://doi.org/10.1080/01621459.1963.10500830

[14] David Kempe, Jon Kleinberg, and Éva Tardos. 2003. Maximizing the spread of influence through a social network. In *Proceedings of the Ninth ACM SIGKDD International Conference on Knowledge Discovery and Data Mining* (Washington, D.C.) *(KDD '03)*. Association for Computing Machinery, New York, NY, USA, 137–146. https://doi.org/10.1145/956750.956769

[15] Samir Khuller, Anna Moss, and Joseph (Seffi) Naor. 1999. The budgeted maximum coverage problem. *Inform. Process. Lett.* 70, 1 (1999), 39–45. https://doi.org/10.1016/S0020-0190(99)00031-9

[16] Nano Langenheim, Marcus White, Nigel Tapper, Stephen J. Livesley, and Diego Ramirez-Lovering. 2020. Right tree, right place, right time: A visual-functional design approach to select and place trees for optimal shade benefit to commuting pedestrians. *Sustainable Cities and Society* 52 (2020), 101816. https://doi.org/10.1016/j.scs.2019.101816

[17] Larry J. LeBlanc and David E. Boyce. 1986. A bilevel programming algorithm for exact solution of the network design problem with user-optimal flows. *Transportation Research Part B: Methodological* 20, 3 (1986), 259–265. https://doi.org/10.1016/0191-2615(86)90021-4

[18] Haiwei Li, Yongling Zhao, Chenghao Wang, Diana Ürge-Vorsatz, Jan Carmeliet, and Ronita Bardhan. 2024. Cooling efficacy of trees across cities is determined by background climate, urban morphology, and tree trait. *Communications Earth & Environment* 5, 1 (2024), 754. https://doi.org/10.1038/s43247-024-01908-4

[19] Ruolin Li, Negar Mehr, and Roberto Horowitz. 2023. Submodularity of optimal sensor placement for traffic networks. *Transportation Research Part B: Methodological* 171 (2023), 29–43. https://doi.org/10.1016/j.trb.2023.02.008

[20] Yuchen Li, Ju Fan, Yanhao Wang, and Kian-Lee Tan. 2018. Influence Maximization on Social Graphs: A Survey. *IEEE Transactions on Knowledge and Data Engineering* 30, 10 (Oct 2018), 1852–1872. https://doi.org/10.1109/TKDE.2018.2807843

[21] Zheng Li and Jun Ma. 2022. Discussing street tree planning based on pedestrian volume using machine learning and computer vision. *Building and Environment* 219 (2022), 109178. https://doi.org/10.1016/j.buildenv.2022.109178

[22] Wei Lu, Wei Chen, and Laks V. S. Lakshmanan. 2015. From competition to complementarity: comparative influence diffusion and maximization. *Proc. VLDB Endow.* 9, 2 (Oct. 2015), 60–71. https://doi.org/10.14778/2850578.2850581

[23] Brian J. Lunday. 2024. The maximal covering location disruption problem. *Computers & Operations Research* 169 (2024), 106721. https://doi.org/10.1016/j.cor.2024.106721

[24] Sepideh Mahabadi, Sherry Sarkar, and Jakub Tarnawski. 2026. Improved Algorithms for Fair Matroid Submodular Maximization. In *The Thirty-ninth Annual Conference on Neural Information Processing Systems*. https://openreview.net/forum?id=gfGLEtWAqV

[25] Vladimir Marianov and H.A. Eiselt. 2024. Fifty Years of Location Theory - A Selective Review. *European Journal of Operational Research* 318, 3 (2024), 701–718. https://doi.org/10.1016/j.ejor.2024.01.036

[26] Nimrod Megiddo, Eitan Zemel, and S. Louis Hakimi. 1983. The Maximum Coverage Location Problem. *SIAM Journal on Algebraic Discrete Methods* 4, 2 (1983), 253–261. https://doi.org/10.1137/0604028 arXiv:https://doi.org/10.1137/0604028

[27] Negar Mehr and Roberto Horowitz. 2018. A Submodular Approach for Optimal Sensor Placement in Traffic Networks. In *2018 Annual American Control Conference (ACC)*. IEEE, Milwaukee, WI, USA, 6353–6358. https://doi.org/10.23919/ACC.2018.8431678

[28] George L Nemhauser, Laurence A Wolsey, and Marshall L Fisher. 1978. An analysis of approximations for maximizing submodular set functions—I. *Mathematical programming* 14 (1978), 265–294. https://doi.org/10.1007/BF01588971

[29] Iasonas Nikolaou, Miltiadis Stouras, Stratis Ioannidis, and Evimaria Terzi. 2025. Online Two-Stage Submodular Maximization. In *Advances in Neural Information Processing Systems*, D. Belgrave, C. Zhang, H. Lin, R. Pascanu, P. Koniusz, M. Ghassemi, and N. Chen (Eds.), Vol. 38. Curran Associates, Inc., 2128–2167.

[30] Tareq Si Salem, Gözde Özcan, Iasonas Nikolaou, Evimaria Terzi, and Stratis Ioannidis. 2024. Online Submodular Maximization via Online Convex Optimization. *Proceedings of the AAAI Conference on Artificial Intelligence* 38, 13 (Mar. 2024), 15038–15046. https://doi.org/10.1609/aaai.v38i13.29425

[31] Shaojie Tang and Jing Yuan. 2024. Non-monotone Sequential Submodular Maximization. *Proceedings of the AAAI Conference on Artificial Intelligence* 38, 14 (Mar. 2024), 15284–15291. https://doi.org/10.1609/aaai.v38i14.29452

[32] Pieter Vansteenwegen, Wouter Souffriau, and Dirk Van Oudheusden. 2011. The orienteering problem: A survey. *European Journal of Operational Research* 209, 1 (2011), 1–10. https://doi.org/10.1016/j.ejor.2010.03.045

[33] Qunshan Zhao, Elizabeth A. Wentz, and Alan T. Murray. 2017. Tree shade coverage optimization in an urban residential environment. *Building and Environment* 115 (2017), 269–280. https://doi.org/10.1016/j.buildenv.2017.01.036

A Missing Proofs

In this section, we present the missing proofs in this paper.

Proof of Theorem 4.1. We reduce from MCP on a 4-neighbor grid under the L_1 metric with radius D and budget k. An instance specifies a finite set of weighted demand nodes $Q \subseteq W$ with weights $\{w_q\}_{q \in Q}$ and a finite set of candidate centers $T \subseteq W$. The goal is to choose $S \subseteq T$ with $|S| \leq k$ to maximize $\sum_{q \in Q} w_q \cdot \mathbb{I}[\exists u \in S : d(u,q) \leq D]$, where $d(\cdot,\cdot)$ is the Manhattan hop distance. This problem is NP-hard.

We build a RUTO instance as follows. Set the kernel to hard-cutoff $K(u,v) = \mathbb{I}[d(u,v) \leq D]$, the transform $\phi(t) = \min\{t, 1\}$, and $\lambda = 0$. Let the plantable set equal the allowed centers T. Define the OD distribution μ to be supported on the pairs (q,q) for all $q \in Q$ with probabilities $p_q \coloneqq w_q / \sum_{q' \in Q} w_{q'}$. Enforce a corridor of width $r = 0$ for each (q,q) so that the unique admissible path is the length-zero path at node q.

For any selection $S \subseteq T$ with $|S| \leq k$, the best-response path at OD (q,q) yields utility $g_P(S) = \phi(h_q(S)) = \min\{\sum_{u \in S} \mathbb{I}[d(u,q) \leq D], 1\}$. Therefore the RUTO objective equals $\sigma(S) = \sum_{q \in Q} p_q \cdot \min\{h_q(S), 1\} = \frac{1}{\sum_{q'} w_{q'}} \sum_{q \in Q} w_q \cdot \mathbb{I}[\exists u \in S : d(u,q) \leq D]$. Maximizing $\sigma(S)$ under $|S| \leq k$ is thus equivalent to the original maximum-coverage objective up to a positive normalization, and the reduction is polynomial in input size.

Hence, RUTO is NP-hard. □

Proof of Lemma 4.2. Consider a single OD-pair (s,t) so that μ is a point mass. Let $\phi(t) = t$, $\lambda = \frac{3}{4}$, and use a hard-cutoff kernel with radius $D = 0$ so a tree contributes only at its own node. Construct two simple $s \to t$ paths in G_{walk}: a baseline path B of length 2 visiting no plantable nodes, and a detour path D of length 4 visiting two plantable nodes a and b in sequence and no other plantable nodes. Let the candidate set be $T = \{a, b\}$.

For $S = \emptyset$: $g_B = -2\lambda = -\frac{3}{2}$ and $g_D = -4\lambda = -3$, hence B is chosen and $\sigma(\emptyset) = -\frac{3}{2}$. For $S = \{a\}$ or $S = \{b\}$: $g_B = -\frac{3}{2}$ and $g_D = 1 - 4\lambda = 1 - 3 = -2$, hence B is chosen and $\sigma(\{a\}) = \sigma(\{b\}) = -\frac{3}{2}$. For $S = \{a, b\}$: $g_B = -\frac{3}{2}$ and $g_D = 2 - 4\lambda = 2 - 3 = -1$, hence D is chosen and $\sigma(\{a, b\}) = -1$.

Let $A = \emptyset$, $B' = \{a\}$, and $x = b$. Then $\sigma(A \cup \{x\}) - \sigma(A) = \sigma(\{b\}) - \sigma(\emptyset) = (-\frac{3}{2}) - (-\frac{3}{2}) = 0$, whereas $\sigma(B' \cup \{x\}) - \sigma(B') = \sigma(\{a, b\}) - \sigma(\{a\}) = -1 - (-\frac{3}{2}) = \frac{1}{2}$. The marginal gain increases with the superset, violating diminishing returns, so $\sigma(\cdot)$ is non-submodular. □

Correctness of Eq. (2). For the upper-bound branch, let S°_{ub} be the optimal solution of $\text{UB}(\cdot)$ under $|S| \le k$. Recall that $\text{UB}(\cdot)$ is monotone and submodular, and that S_{ub} is its greedy maximizer. Applying the $(1-1/e)$ greedy guarantee [28] to the baseline-shifted objective $\text{UB}(S) - \text{UB}(\emptyset)$ yields

$$\text{UB}(S_{\text{ub}}) - \text{UB}(\emptyset) \;\ge\; (1 - 1/e)\left(\text{UB}(S^\circ_{\text{ub}}) - \text{UB}(\emptyset)\right).$$

Applying $\text{UB}(S^\circ_{\text{ub}}) \ge \text{UB}(S^\circ)$, multiplying both sides by the non-negative factor $\frac{\sigma(S_{\text{ub}})-\text{UB}(\emptyset)}{\text{UB}(S_{\text{ub}})-\text{UB}(\emptyset)}$ and using $\sigma(\cdot) \le \text{UB}(\cdot)$ at S°, we obtain

$$\sigma(S_{\text{ub}}) - \text{UB}(\emptyset) \ge \frac{\sigma(S_{\text{ub}}) - \text{UB}(\emptyset)}{\text{UB}(S_{\text{ub}}) - \text{UB}(\emptyset)}(1 - 1/e)\left(\sigma(S^\circ) - \text{UB}(\emptyset)\right).$$

Since $\text{UB}(\emptyset) \ge \text{LB}(\emptyset)$, adding $\text{UB}(\emptyset)$ to left side and $\text{LB}(\emptyset)$ to right side gives the upper-bound branch in Eq. (2):

$$\sigma(S_{\text{ub}}) \ge \frac{\sigma(S_{\text{ub}}) - \text{UB}(\emptyset)}{\text{UB}(S_{\text{ub}}) - \text{UB}(\emptyset)}(1 - 1/e)\left(\sigma(S^\circ) - \text{UB}(\emptyset)\right) + \text{LB}(\emptyset). \tag{10}$$

For the lower-bound branch, let S°_{lb} be the optimal solution of LB under $|S| \le k$. Recall that S_{lb} is the greedy maximizer of $\text{LB}(\cdot)$, which is monotone and submodular. Applying the $(1 - 1/e)$ guarantee to $\text{LB}(S) - \text{LB}(\emptyset)$ gives

$$\text{LB}(S_{\text{lb}}) - \text{LB}(\emptyset) \ge (1 - 1/e)\left(\text{LB}(S^\circ_{\text{lb}}) - \text{LB}(\emptyset)\right).$$

Using $\sigma(\cdot) \ge \text{LB}(\cdot)$ at S_{lb}, $\text{LB}(S^\circ_{\text{lb}}) \ge \text{LB}(S^\circ)$ and rearranging,

$$\begin{aligned}\sigma(S_{\text{lb}}) \ge& (1 - 1/e)\left(\text{LB}(S^\circ) - \text{LB}(\emptyset)\right) + \text{LB}(\emptyset)\\ =& \frac{\text{LB}(S^\circ) - \text{LB}(\emptyset)}{\sigma(S^\circ) - \text{UB}(\emptyset)}(1 - 1/e)\left(\sigma(S^\circ) - \text{UB}(\emptyset)\right) + \text{LB}(\emptyset).\end{aligned} \tag{11}$$

Thus, the lower-bound branch in Eq. (2) holds as stated.

Recall that S_{sand} is the better of S_{ub} and S_{lb}, i.e., $\sigma(S_{\text{sand}}) = \max\{\sigma(S_{\text{ub}}), \sigma(S_{\text{lb}})\}$. Substituting Eq. (10) and Eq. (11) into this identity yields Eq. (2). □

Proof of Lemma 5.1. Define $F_{s,t}(S) \coloneqq \sum_{v \in U(s,t)} \phi(h_v(S))$. Fix (s,t). Since each $h_v(\cdot)$ is modular and nondecreasing in S, and since ϕ is concave and nondecreasing with $\phi(0) = 0$, the concave-over-modular property implies that $S \mapsto \phi(h_v(S))$ is submodular and nondecreasing.[3] Summing over $v \in U(s,t)$ preserves these properties, hence $F_{s,t}$ is submodular and nondecreasing.

For UB_1, recall Eq. (3), we have $\text{UB}_1(S) = \mathbb{E}_{(s,t)\sim\mu}\left[F_{s,t}(S) - \lambda C_{\min}(s,t)\right]$, where $C_{\min}(s,t)$ is constant in S and thus does not affect submodularity, while taking expectations preserves it. Therefore, $\text{UB}_1(S)$ is submodular.

For UB_2, $B(s,t)$, defined in Section 5.1.2, is constant in S. Since $F_{s,t}(S)$ is nondecreasing and submodular, the truncation $S \mapsto \min\{B(s,t), F_{s,t}(S)\}$ is also nondecreasing and submodular. Expectation over $(s,t) \sim \mu$ and subtracting the constant $\lambda C_{\min}(s,t)$ preserve submodularity, hence $\text{UB}_2(S)$ is submodular. □

Proof of Lemma 5.2. Fix (s,t). For any admissible path P, we have $P \subseteq U(s,t)$, hence $\sum_{v\in P} \phi(h_v(S)) \le \sum_{v \in U(s,t)} \phi(h_v(S))$. Moreover, $h_v(S) \le k$ since $0 \le K(u,v) \le 1$ and $|S| \le k$, so $\sum_{v\in P} \phi(h_v(S)) \le |P| \cdot \phi(k) \le B(s,t)$. We also have $\text{cst}(P) \ge C_{\min}(s,t)$. Therefore for any P,

$$g(S;P) \le \min\{B(s,t), \sum_{v \in U(s,t)} \phi(h_v(S))\} - \lambda C_{\min}(s,t).$$

Taking the maximum over $P \in \mathcal{P}(s,t)$ and then the expectation over $(s,t) \sim \mu$ yields $\sigma(S) \le \text{UB}_2(S)$. Finally,

$$\min\{B(s,t), \sum_{v \in U(s,t)} \phi(h_v(S))\} \le \sum_{v \in U(s,t)} \phi(h_v(S)),$$

thus $\text{UB}_2(S) \le \text{UB}_1(S)$. □

Proof of Lemma 5.3. For LB_1, fix (s,t) and recall that $P^{\text{sel}}(s,t)$ is the prescribed admissible path (see Section 5.1.3). For any node v the set function $S \mapsto \phi(h_v(S))$ is submodular and nondecreasing by the concave-over-modular property. Hence $S \mapsto \sum_{v \in P^{\text{sel}}(s,t)} \phi(h_v(S))$ is submodular and nondecreasing since sums preserve these properties. The term $-\lambda\, \text{cst}(P^{\text{sel}}(s,t))$ is constant with respect to S, so it does not affect submodularity. Therefore the integrand $g(S; P^{\text{sel}}(s,t))$ is submodular in S for each (s,t). Expectation over $(s,t) \sim \mu$ preserves submodularity, hence $\text{LB}_1(S)$ is submodular.

For LB_2, fix (s,t) and a distribution $q_{s,t}$ on $\mathcal{P}(s,t)$ that does not depend on S. As above, $S \mapsto g(S;P)$ is submodular for every admissible path P, and $-\lambda\, \text{cst}(P)$ is constant in S. Thus $S \mapsto \mathbb{E}_{P\sim q_{s,t}}[g(S;P)]$ is submodular because nonnegative weighted sums (i.e., expectations) of submodular functions are still submodular. Taking expectation over $(s,t) \sim \mu$ preserves submodularity, so $\text{LB}_2(S)$ is submodular.

For $\text{LB}_3^{(i)}$, fix i and note that $q_{s,t}^{(S_i)}$ is a distribution on $\mathcal{P}(s,t)$ that is fixed with respect to S. By the same reasoning as for LB_2, $S \mapsto \mathbb{E}_{P\sim q_{s,t}^{(S_i)}}[g(S;P)]$ is submodular for each (s,t), and the outer expectation over $(s,t) \sim \mu$ preserves submodularity. Hence $\text{LB}_3^{(i)}(S)$ is submodular for any fixed i. □

Proof of Lemma 5.4. Fix (s,t). For any admissible path $P \in \mathcal{P}(s,t)$ we have $g(S;P) \le \max_{P' \in \mathcal{P}(s,t)} g(S;P')$. Hence

$$g(S; P^{\text{sel}}(s,t)) \le \max_{P' \in \mathcal{P}(s,t)} g(S;P').$$

[3] Here $S \mapsto \phi(h_v(S))$ denotes the set function $f_v : 2^T \to \mathbb{R}_{\ge 0}$ defined by $f_v(S) = \phi(h_v(S))$.

Taking expectation over $(s,t) \sim \mu$ yields $\mathrm{LB}_1(S) \le \sigma(S)$.

For LB_2, recall that $q_{s,t}$ is a distribution on $\mathcal{P}(s,t)$ that does not depend on S. By the same pointwise inequality and linearity of expectation we obtain

$$\mathbb{E}_{P\sim q_{s,t}}[g(S;P)] \le \max_{P'\in\mathcal{P}(s,t)} g(S;P').$$

Taking expectation over $(s,t) \sim \mu$ gives $\mathrm{LB}_2(S) \le \sigma(S)$.

For $\mathrm{LB}_3^{(i)}$, with i fixed the mixture $q_{s,t}^{(S_i)}$ is fixed with respect to S. Applying the same argument yields

$$\mathbb{E}_{P\sim q_{s,t}^{(S_i)}}[g(S;P)] \le \max_{P'\in\mathcal{P}(s,t)} g(S;P'),$$

and then $\mathrm{LB}_3^{(i)}(S) \le \sigma(S)$ after expectation over $(s,t) \sim \mu$. □

B Sample Size Selection

Let $Y(S;(s,t))$ denote the per-OD-pair integrand of a chosen objective, and let $\widehat{Y}_m(S) = \frac{1}{m}\sum_{i=1}^{m} Y\big(S;(s_i,t_i)\big)$ with i.i.d. $(s_i,t_i) \sim \mu$. Assume bounds $a \le Y(S;(s,t)) \le b$ for all (s,t). Hoeffding's inequality [13] gives

$$\Pr\left(\left|\widehat{Y}_m(S) - \mathbb{E}_{(s,t)\sim\mu}[Y(S;(s,t))]\right| \ge \varepsilon\right) \le 2\exp\left(-\frac{2m\varepsilon^2}{(b-a)^2}\right).$$

Given $\varepsilon > 0$ and $\delta \in (0,1)$, it suffices to take

$$m \ge \frac{(b-a)^2}{2\varepsilon^2}\,\log\frac{2}{\delta}. \tag{12}$$

If uniform accuracy is required over J evaluations, we can replace δ by δ/J via a union bound.

We now give convenient choices of $(b-a)$ that are independent of S. Let $\phi_k := \phi(k)$, $L^\star := \sup_{(s,t)} \max_{P\in\mathcal{P}(s,t)} |P|$, and $C^\star := \sup_{(s,t)} \max_{P\in\mathcal{P}(s,t)} \mathrm{cst}(P)$. For corridor-restricted families one may replace the suprema by corridor-specific bounds. For UB_1 additionally let $U^\star := \sup_{(s,t)} |U(s,t)|$.

All objectives except UB_1 admit the unified bound

$$(b-a) \;\le\; L^\star\phi_k + \lambda C^\star,$$

since their per–OD integrands are of the form $\sum_{v\in P}\phi(h_v(S)) - \lambda\,\mathrm{cst}(P)$ (or mixtures thereof), with $\sum_{v\in P}\phi(h_v(S)) \le |P|\phi_k \le L^\star\phi_k$ and $-\lambda\,\mathrm{cst}(P) \ge -\lambda C^\star$.

For UB_1 we have

$$(b-a) \;\le\; U^\star\phi_k + \lambda C^\star,$$

because $\sum_{v\in U(s,t)}\phi(h_v(S)) \le |U(s,t)|\,\phi_k \le U^\star\phi_k$ and the cost term contributes at most $\lambda C^\star$ to the width.

By applying the bounds above and Eq. (12), we can approximate the objective and its upper and lower bounds (Eqs. (1), (3)–(7)) to within any prescribed $\varepsilon > 0$ with probability at least $1-\delta$.

C Missing Experimental Details

In this section, we introduce the omitted experimental settings and results. For all line plots in this section, we use the same legend as in Figure 2 in the main text.

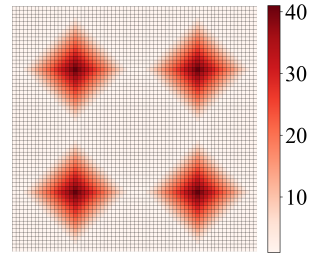
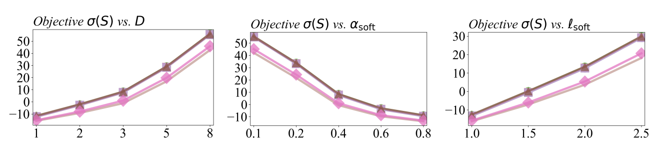


Figure 10: Node-level hotspot weights defining the hotspot-weighted OD distribution μ_{hot} on the 64×64 2D grid.

Varying τ with $I=5$			Varying I with $\tau=1.0$		
τ	Objective	Runtime (sec)	I	Objective	Runtime (sec)
0.5	11.06	215.62	3	11.05	**209.71**
1.0	**11.17**	215.00	5	11.17	213.63
1.5	11.12	**214.47**	10	11.20	226.48
2.0	11.07	214.48	20	**11.25**	252.23

Table 2: Objective value and runtime of the LB_3-based method under different τ and I.



(a) K_{hard} **(b)** K_{exp} **(c)** K_{gauss}

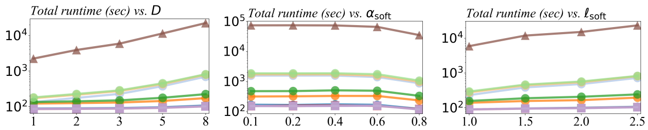
Figure 11: Objective under varying different kernel function parameters.

C.1 Additional Experimental Details on Synthetic Grid Graphs

In this section, we provide additional experimental settings and results corresponding to Sections 6.2.1, 6.2.3, 6.2.6, and 6.2.7, while deferring the details for Sections 6.2.10 and 6.2.11 to the full version of the paper.

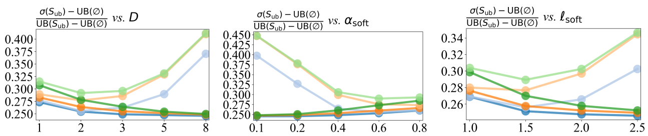
C.1.1 Hotspot-weighted distribution in Section 6.2.1. Figure 10 visualizes the hotspot-weighted distribution μ_{hot} used in our main experiments on the 64×64 grid. Each node v is assigned a non-negative weight w_v, with four hotspot regions receiving elevated weights. The maximum hotspot weight is 40, while the smallest weight is 1. We define μ_{hot} by normalizing these weights, i.e., $\mu_{\mathrm{hot}}(v) \propto w_v$. For each agent, we sample its origin s and destination t independently from μ_{hot}, so OD pairs are concentrated around the hotspot regions rather than being uniformly spread over the grid.

C.1.2 Results in Section 6.2.3. Table 2 reports the results of varying the hyperparameters of LB_3. Changing τ or I has little effect on the objective value. Varying τ has negligible impact on runtime, and increasing I keeps the runtime within the same order of magnitude despite a moderate increase.



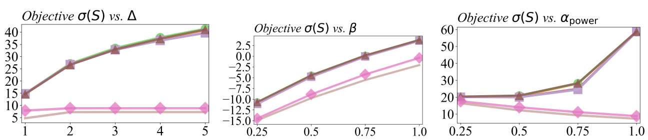
(a) K_{hard} (b) K_{exp} (c) K_{gauss}

Figure 12: Runtime under varying different kernel function parameters.



(a) K_{hard} (b) K_{exp} (c) K_{gauss}

Figure 13: Approximation auxiliary quantity $\frac{\sigma(S_{\text{ub}})-\text{UB}(\emptyset)}{\text{UB}(S_{\text{ub}})-\text{UB}(\emptyset)}$ under varying different kernel function parameters.



(a) ϕ_{clip} (b) ϕ_{expsat} (c) ϕ_{power}

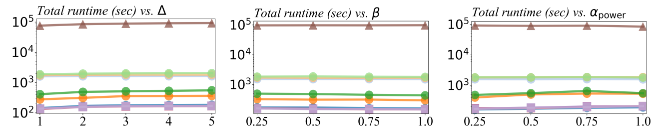
Figure 14: Objective under varying different transformation function parameters.

C.1.3 Results in Section 6.2.6. Figures 11, 12, and 13 present the objective value, runtime, and auxiliary quantity, respectively, under different kernel hyperparameter settings. We use the transform ϕ_{log1p} in these experiments, since its smooth, non-saturating shape (rather than a hard-cutoff) pairs naturally with smoothly decaying shade kernels.

Beyond the comparison among methods, we also observe consistent trends as the effective shading range grows. For the three kernels we consider, this corresponds to larger D for K_{hard}, smaller α_{soft} for K_{exp}, and larger ℓ_{soft} for K_{gauss}. Larger shading ranges lead to higher objective values and longer runtime. Intuitively, when each tree influences a wider area, agents receive more shade and thus higher expected utility, and a larger set of nodes that can affect agents' utilities makes the search for the best k tree locations more expensive.

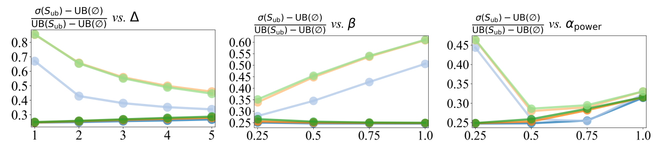
As discussed in the main text, for the UB_2-based algorithms the auxiliary quantity first decreases slightly and then increases as the shading range grows. This suggests that the tightness of the upper bound can be worst at intermediate ranges where the kernel is neither very local nor very global. However, this effect does not harm practical performance, as the objective curves in Figure 11 remain strong for all settings.

C.1.4 Results in Section 6.2.7. Figures 14, 15, and 16 present the objective value, runtime, and auxiliary quantity, respectively, under different transform hyperparameter settings. Overall, consistent



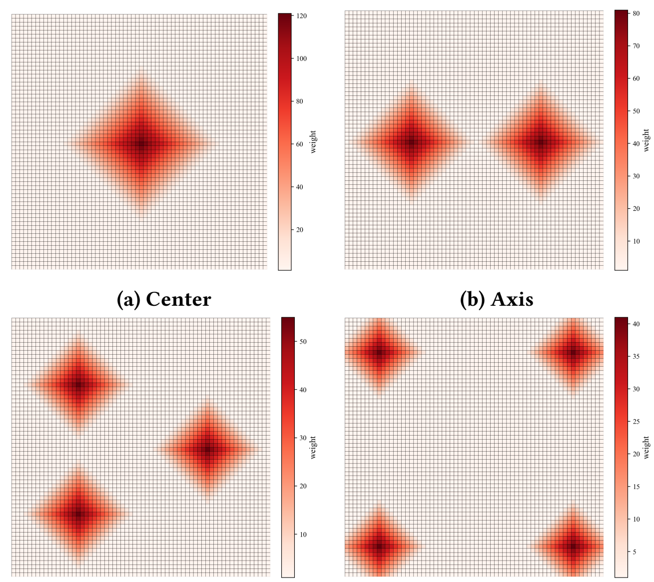
(a) ϕ_{clip} (b) ϕ_{expsat} (c) ϕ_{power}

Figure 15: Runtime under varying different transformation function parameters.



(a) ϕ_{clip} (b) ϕ_{expsat} (c) ϕ_{power}

Figure 16: Approximation auxiliary quantity $\frac{\sigma(S_{\text{ub}})-\text{UB}(\emptyset)}{\text{UB}(S_{\text{ub}})-\text{UB}(\emptyset)}$ under varying different transformation function parameters.

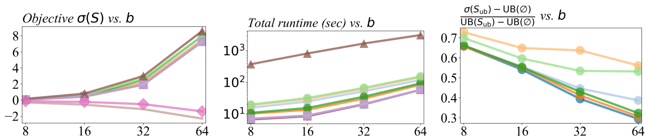


(a) Center **(b) Axis** **(c) Triangle** **(d) Periphery**

Figure 17: Visualizations of four hotspot weight patterns for μ_{hot} on the 64×64 grid.

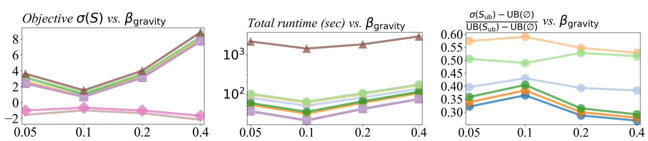
with the main experimental results, our algorithms are both effective and efficient. Our methods are up to two orders of magnitude faster than *Greedy*, and the auxiliary quantity remains at a constant level across all tested settings.

C.1.5 Details and results in Section 6.2.10. First, we describe how we construct the four hotspot-weight patterns. For the *center*, *axis*, *triangle*, and *periphery* configurations, we place 1, 2, 3, and 4 hotspot regions, respectively, and choose their locations so that the hotspots are well separated and form simple, regular geometric shapes when multiple hotspots are present (see Figure 17). We also choose the



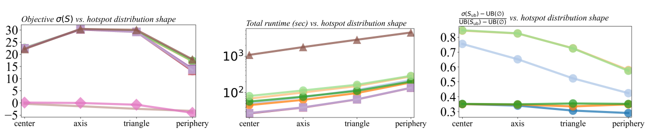
(a) Objective **(b) Runtime** **(c) Quantity**

Figure 18: Objective, runtime, and approximation auxiliary quantity as range b varies under μ_{win}.



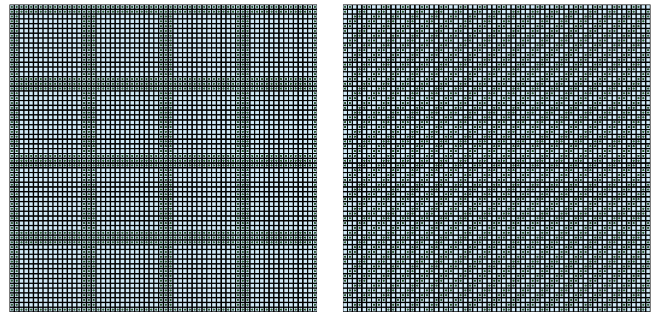
(a) Objective **(b) Runtime** **(c) Quantity**

Figure 19: Objective, runtime, and approximation auxiliary quantity as the β_{gravity} varies under μ_{grav}.

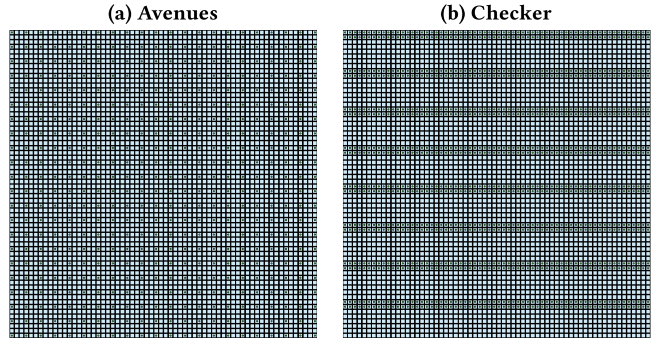


(a) Objective **(b) Runtime** **(c) Quantity**

Figure 20: Objective, runtime, and approximation auxiliary quantity as hotspot pattern varies under μ_{hot}.

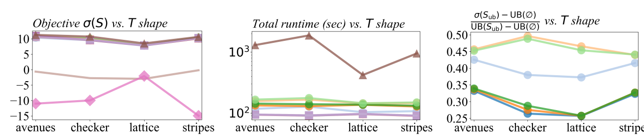


(a) Avenues **(b) Checker**



(c) Lattice **(d) Stripes**

Figure 21: Visualizations of four idealized layouts for the plantable set T: *avenues, checker, lattice,* and *stripes.* Green dots mark plantable candidate locations in T, and blue cells denote walkable nodes in W. By default, we set $W = T$ in our experiments.



(a) Objective **(b) Runtime** **(c) Quantity**

Figure 22: Objective, runtime, and approximation auxiliary quantity under different layouts of the plantable set T.

hotspot weights so that, within each pattern, the product of the number of hotspots and the maximum hotspot weight is approximately equal across patterns, making their overall hotspot intensity comparable.

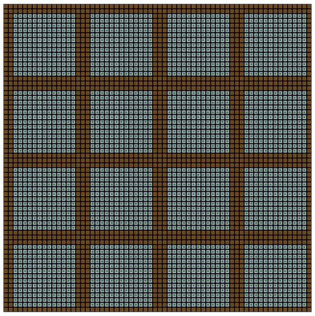
The results are shown in Figures 18, 19, and 20. The comparisons of objective value and runtime across methods are consistent with those in the previous experiments. For the approximation auxiliary quantity, we observe three patterns. Under μ_{win}, it decreases as the window range b increases. Under μ_{grav}, it decreases slightly as β_{gravity} grows, with a small bump around 0.1. Under μ_{hot}, it generally decreases as the number of hotspots increases. In all cases, the auxiliary quantity remains at a constant level and does not affect the effectiveness or efficiency of our SA-based algorithms against the baselines.

C.1.6 Details and results in Section 6.2.11. As introduced in the main text, we study four idealized layouts for the plantable set T, namely *avenues*, *checker*, *lattice*, and *stripes*. These layouts are inspired by common street-tree and planting patterns in real cities and public spaces. The *avenues* layout represents rows of trees aligned with orthogonal city blocks, mimicking regular street grids. The *checker* layout models diagonally alternating plantable cells, similar to artificial plantations with staggered planting. The *lattice* layout captures regularly spaced planting points that maintain a minimum distance between trees, as in plazas or open squares. The *stripes* layout represents long bands of trees along major corridors, such as rows of street trees lining wide avenues. Figure 21 visualizes these four patterns.

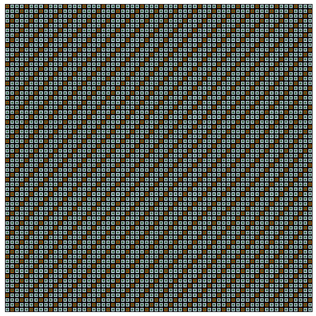
The results for varying the plantable layout T are shown in Figure 22. The comparisons of objective value, runtime, and auxiliary quantity across methods are consistent with the trends observed in the main experiments.

As a complement to varying the plantable set T, we also study different layouts of the walkable set W while keeping T fixed. We consider four stylized patterns, referred to as *avenues*, *checker*, *center*, and *stripes*. These names describe the geometry of the non-walkable region $V \setminus W$, rather than the shape of W itself. Figure 23 visualizes the corresponding layouts.

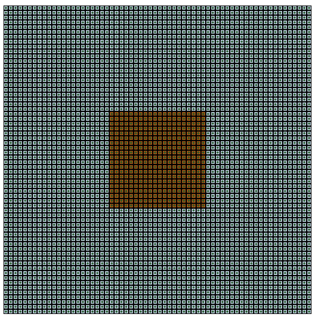
The results for varying the walkable layout W are shown in Figure 24. The comparisons of objective value, runtime, and auxiliary



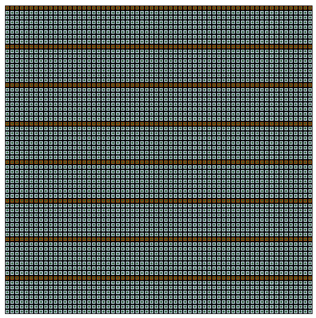
(a) Avenues



(b) Checker

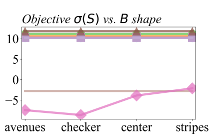


(c) Center

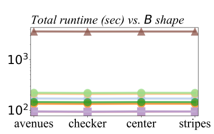


(d) Stripes

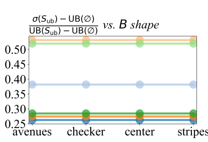
Figure 23: Visualizations of four stylized layouts for the walkable set W and obstacle region $V \setminus W$: *avenues, checker, center,* and *stripes*. Green dots mark plantable candidate locations in T, blue cells denote walkable nodes in W, and brown cells denote non-walkable nodes in $V \setminus W$ (e.g., buildings or other barriers).



(a) Objective



(b) Runtime



(c) Quantity

Figure 24: Objective, runtime, and approximation auxiliary quantity under different layouts of the walkable set W.

quantity across methods are consistent with the conclusions from the main experiments, so we do not repeat the discussion here.

C.2 Missing Experimental Details on Real-World Pedestrian Networks

For the real-world pedestrian networks, we use the same modeling choices as in Section 6.3, with one modification to the per-node benefit transform. In these experiments we use the clip transform ϕ_{clip} with cap $C = 5.0$, rather than the default $C = 2.0$ used on the synthetic grids. The intuition is that in dense tropical cities overlapping tree canopies can yield a larger perceived gain in thermal comfort, and the larger cap also avoids numerically small objective values on much larger graphs. All other hyperparameters follow the default settings described in the main text.

City	Central area	$\|V\|$	$\|W\|$	$\|T\|$
Lima	Gamarra	650,436	283,485	112,762
Dhaka	Motijheel	665,852	158,672	76,113
Jakarta	Tanah Abang	644,800	226,855	104,499
Cairo	Downtown	660,965	288,919	109,754

Table 3: Basic statistics of the real-world pedestrian networks. For each city we report the total number of grid nodes $|V|$, walkable nodes $|W|$, plantable candidates $|T|$, and the central pedestrian area used to define the map window. For each central area, we query OSM with its name, take the returned center point, and expand 2 km in each cardinal direction, yielding an approximately 4 km × 4 km region.

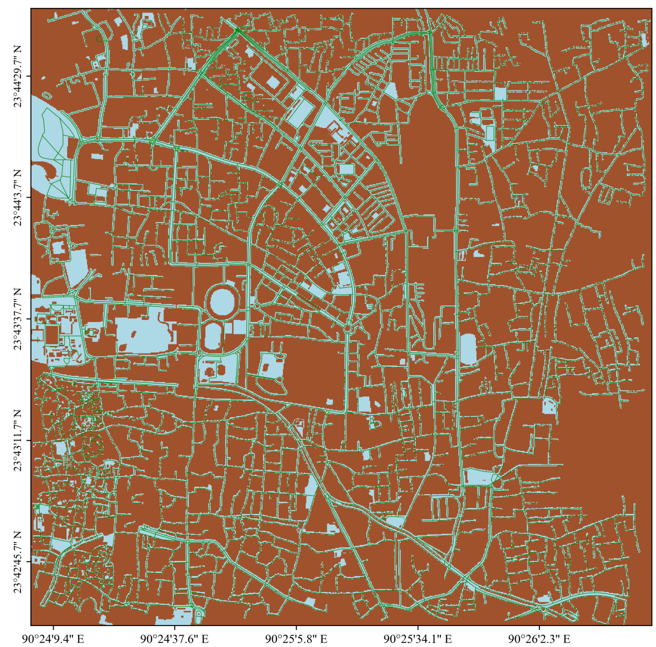


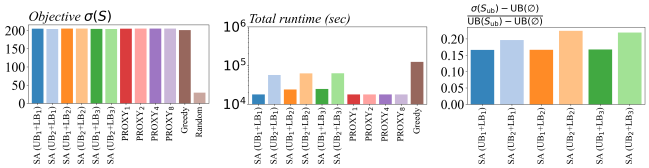
Figure 25: Rasterized pedestrian network in Dhaka.

Table 3 summarizes basic statistics of the four city networks, including the total number of grid nodes $|V|$, walkable nodes $|W|$, and plantable candidates $|T|$.

Figure 8a in the main text illustrates the rasterized pedestrian network and plantable set for Lima. For completeness, Figures 25, 26, and 27 show analogous visualizations for Dhaka, Jakarta, and Cairo, respectively, using the same color coding for buildings, pedestrian routes and walkable areas as in Figure 8a.

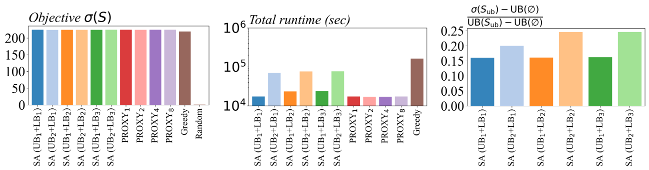
The empirical results on real-world pedestrian networks are summarized in Figures 9, 28, 29, and 30. Across all four cities, the comparisons of objective value, runtime, and approximation auxiliary quantity between our methods and the baselines are consistent with the conclusions drawn from the synthetic-grid experiments, so we do not repeat a detailed discussion here.

These empirical studies confirm that our algorithms can be applied to large, high-resolution urban pedestrian networks while retaining good effectiveness and efficiency. If future work further



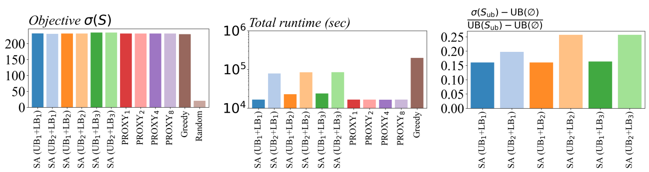
(a) Objective (b) Runtime (c) Quantity

Figure 28: Objective, runtime, and approximation auxiliary quantity in the real-world case study on Dhaka.



(a) Objective (b) Runtime (c) Quantity

Figure 29: Objective, runtime, and approximation auxiliary quantity in the real-world case study on Jakarta.



(a) Objective (b) Runtime (c) Quantity

Figure 30: Objective, runtime, and approximation auxiliary quantity in the real-world case study on Cairo.

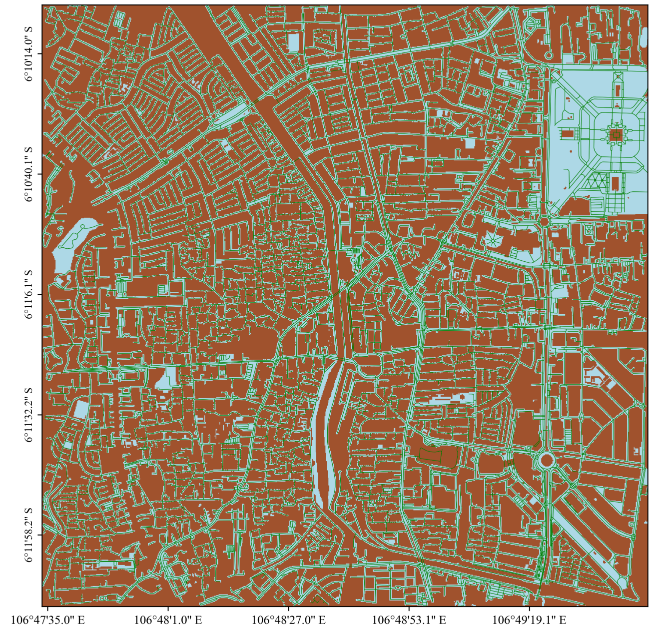


Figure 26: Rasterized pedestrian network in Jakarta.

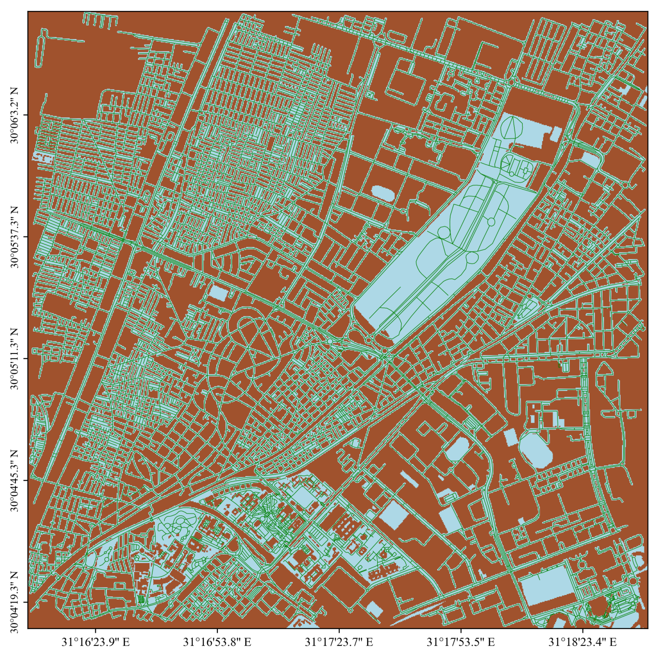


Figure 27: Rasterized pedestrian network in Cairo.

calibrates the RUTO hyperparameters using empirically validated models of shade, thermal comfort, and pedestrian behavior from environmental science, our framework could provide a practical algorithmic basis for budget-constrained street-tree placement in real cities.